

PRESS RECORD

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

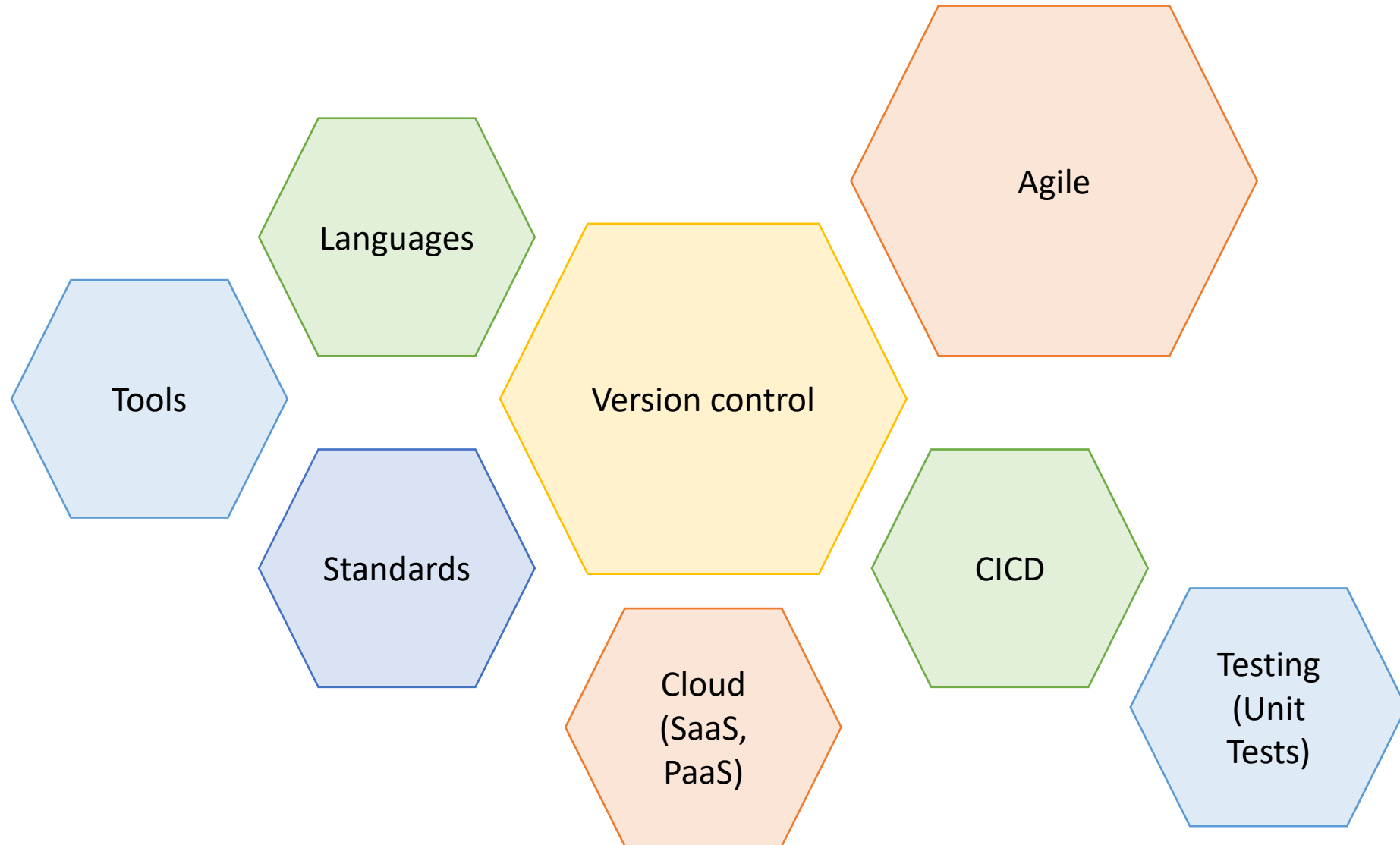
Git and GitLab

COMPX₃₄₁

Before we start ...

- FYI – Industry Input
- FYI – Guest Speakers
- FYI – Soft Schedule

FYI – Industry Input



FYI – Guest Speakers



Sarah Mann
Junior Software developer
NetValue Ltd



Liam Clow
DevOps Engineer
Hamilton City Council

FYI – Soft Schedule

	Lecture 1	Lecture 2	Lecture 3	Assessments
Week 7	Testing w Steve	ANZAC Day	Construction Git and GitLab	
Week 8	Construction Life Cycle Models	Construction Software Quality	Guest Speaker Sarah Mann	CW2 Due CW3 Released
Week 9	Construction Code Quality	Construction Tools and Example	Maintenance An Introduction	
Week 10	Maintenance Analysis and Testing	Maintenance Tools and Example	DevOps An Introduction	CW3 Due CW4 Released
Week 11	DevOps Measuring and Embracing Risk	DevOps CICD Example	Guest Speaker Liam Clow	
Week 12	TBD	TBD	TEST 2	CW4 Due

Git and GitLab

COMPX₃₄₁

Overview

- Version Control
- Git
- GitLab
- UoW GitLab
- Example

Version Control

Version Control

What is Version Control?

- “Version control, also known as source control, is the practice of tracking and managing changes to software code.” ([Bitbucket](#))
- “Version control software keeps track of every modification to the code in a special kind of database. If a mistake is made, developers can turn back the clock and compare earlier versions of the code to help fix the mistake while minimizing disruption to all team members.” ([Bitbucket](#))
- “Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.” ([Git](#))

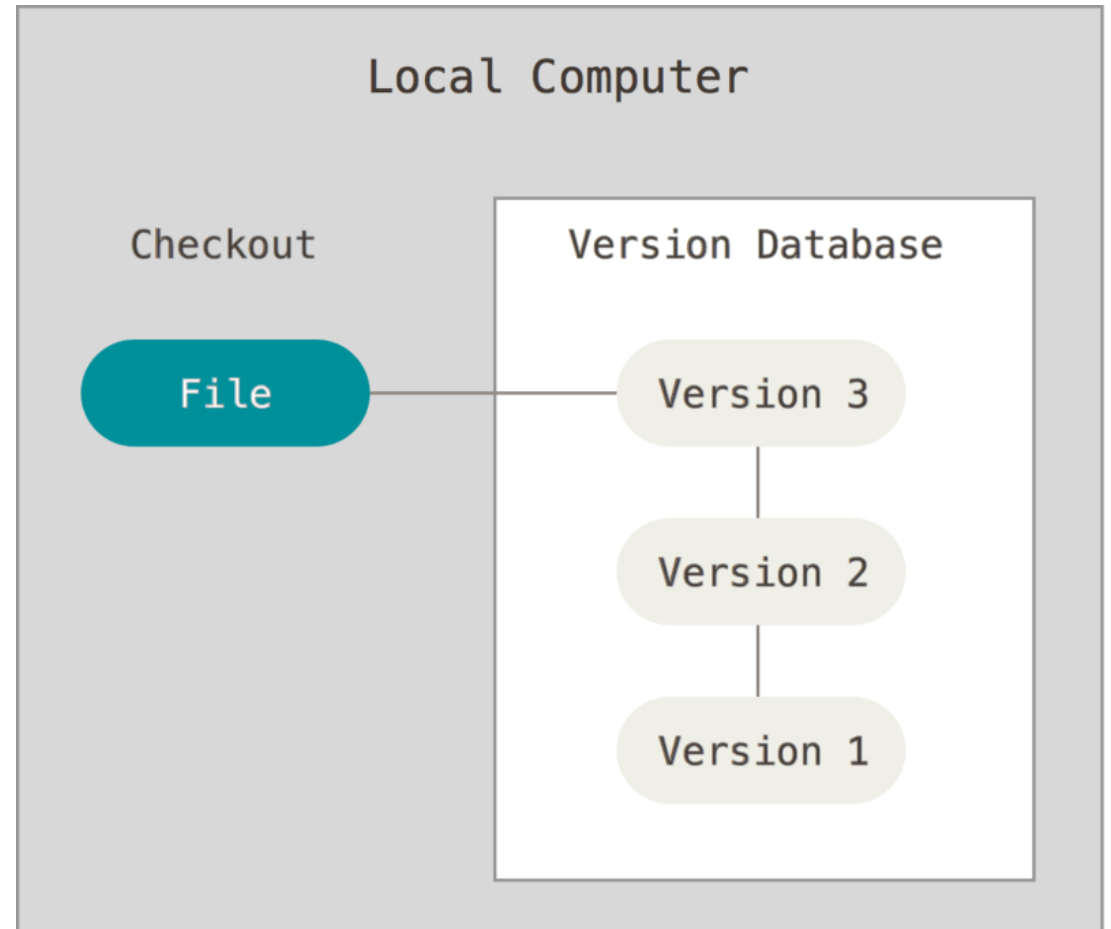
Version Control

Types of Version Control Systems

- Local version control
- Centralized version control
- Distributed version control

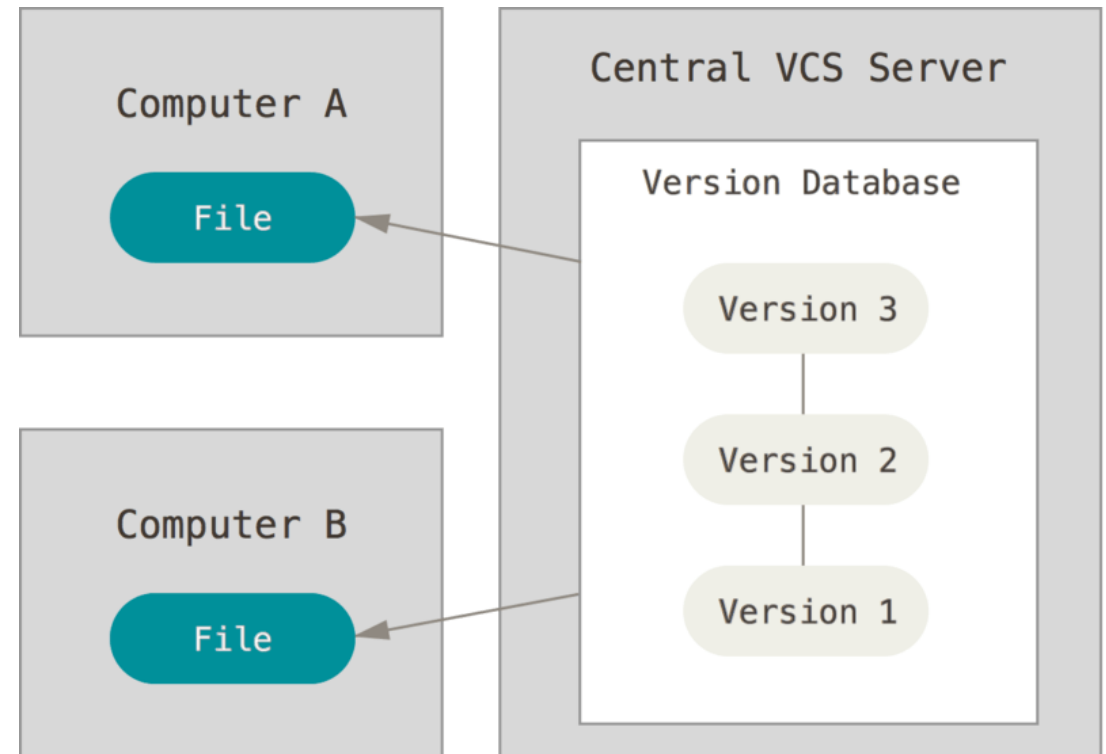
Local Version Control Systems

- Uses a (simple) local database
- Database keeps all file changes
- Called revision control
- Examples: RCS
- ([Git](#))



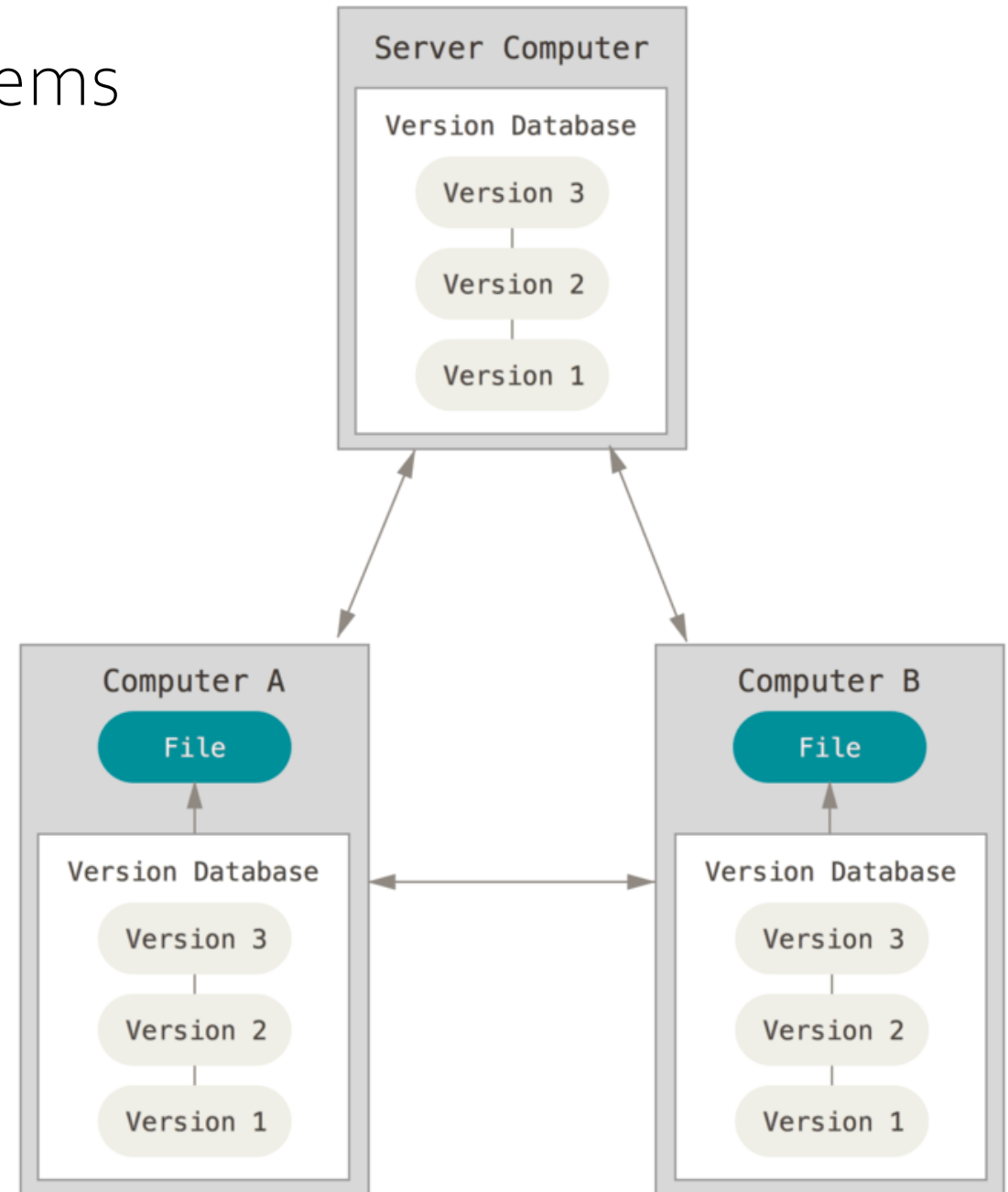
Centralized Version Control Systems

- Enables collaboration
- Single server that holds all versioned files
- Clients “check out” files from the server
- Examples: CVS, Subversion (SVN), and Perforce
- ([Git](#))



Distributed Version Control Systems

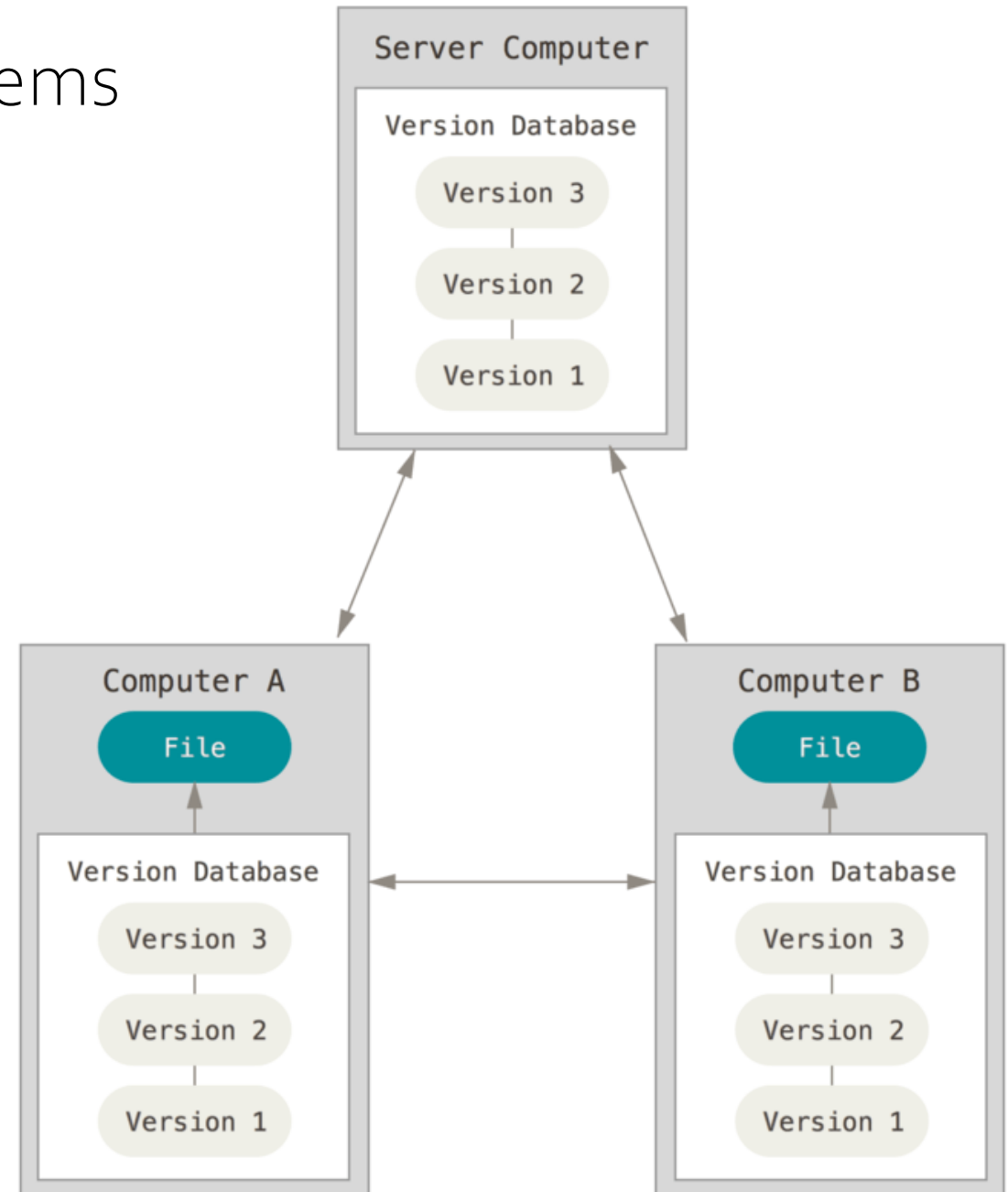
- Client's "clone" the whole repository, including its history.
- Every clone is a full backup of all the data.
- Examples: Git, Mercurial, Bazaar or Darcs
- ([Git](#))



Git

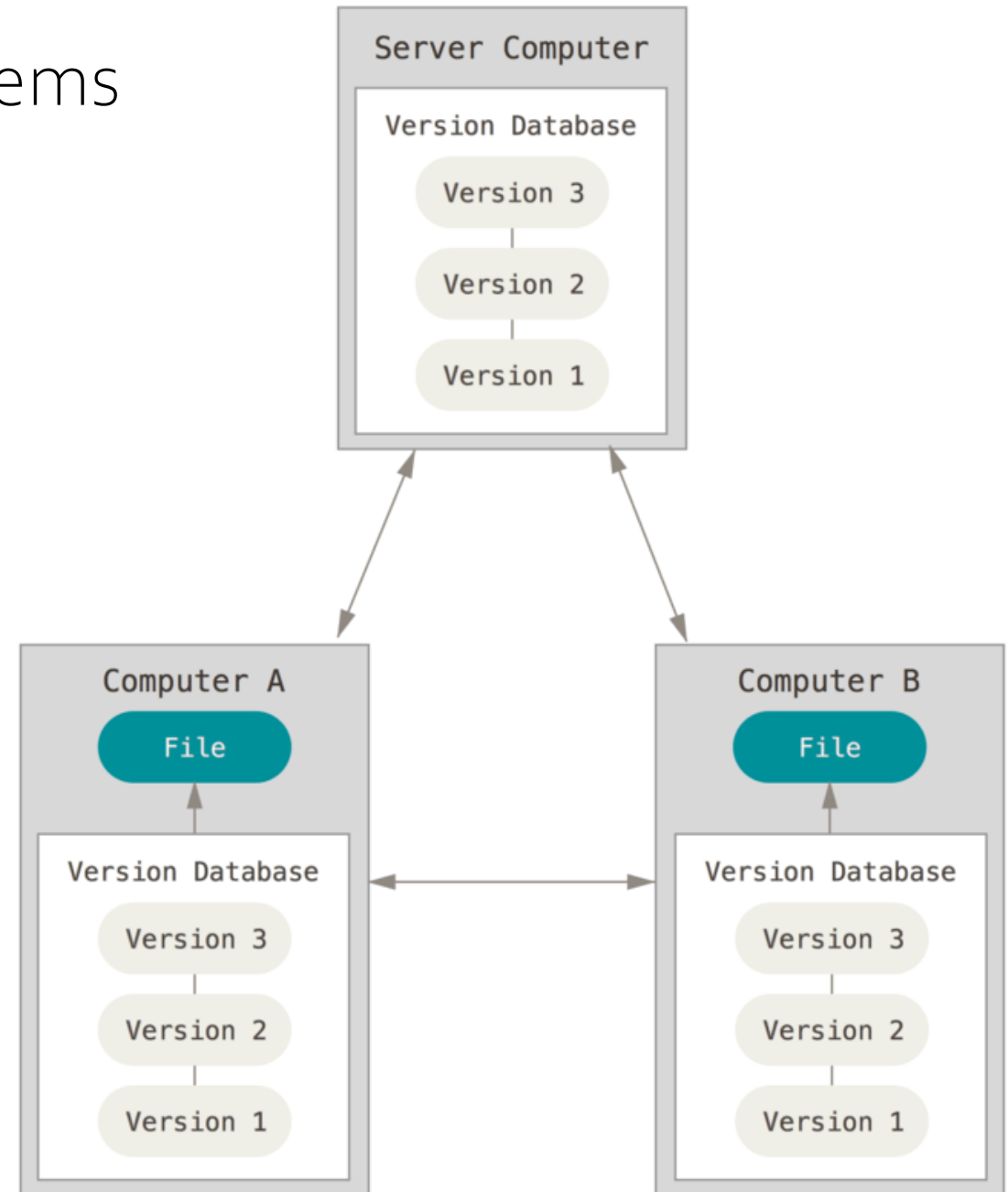
Distributed Version Control Systems

- Client's "clone" the whole repository, including its history.
- Every clone is a full backup of all the data.
- Examples: Git, Mercurial, Bazaar or Darcs
- ([Git](#))



Distributed Version Control Systems

- Client's “clone” the whole repository, including its history.
- Every clone is a full backup of all the data.
- Examples: **Git**, Mercurial, Bazaar or Darcs
- ([Git](#))



Git Stages

Git has three main stages: modified, staged, and committed

Modified:

- You have changed the file but have not staged or commit it yet

Staged:

- You have marked a modified file to go into your next commit

Committed:

- The data has been safely stored in your local database

Working tree, staging area, and Git directory

Working tree:

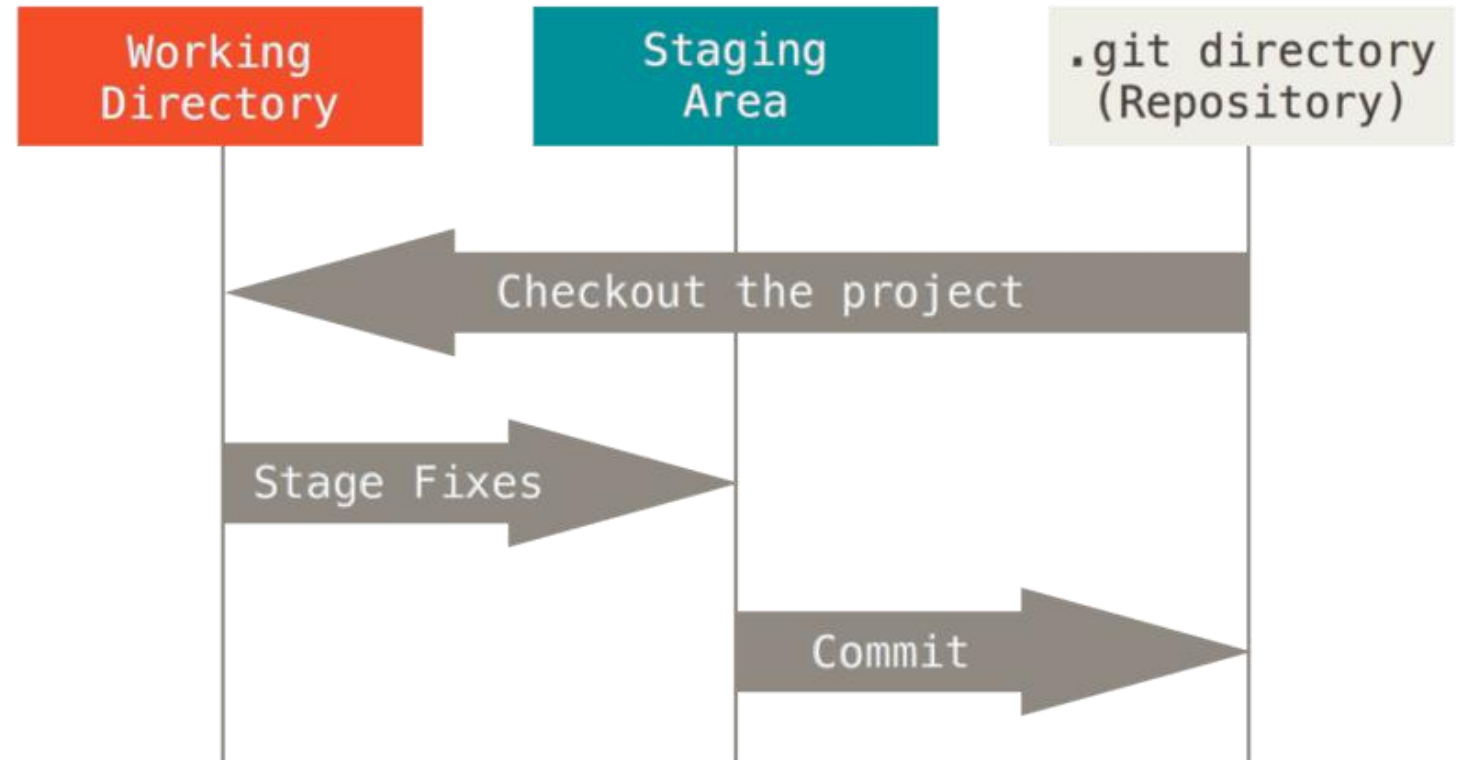
- Single checkout of one version of the project.
- Stored locally on your computer

Staging Area

- A file stored in your Git directory
- Stores info about what goes in your next commit

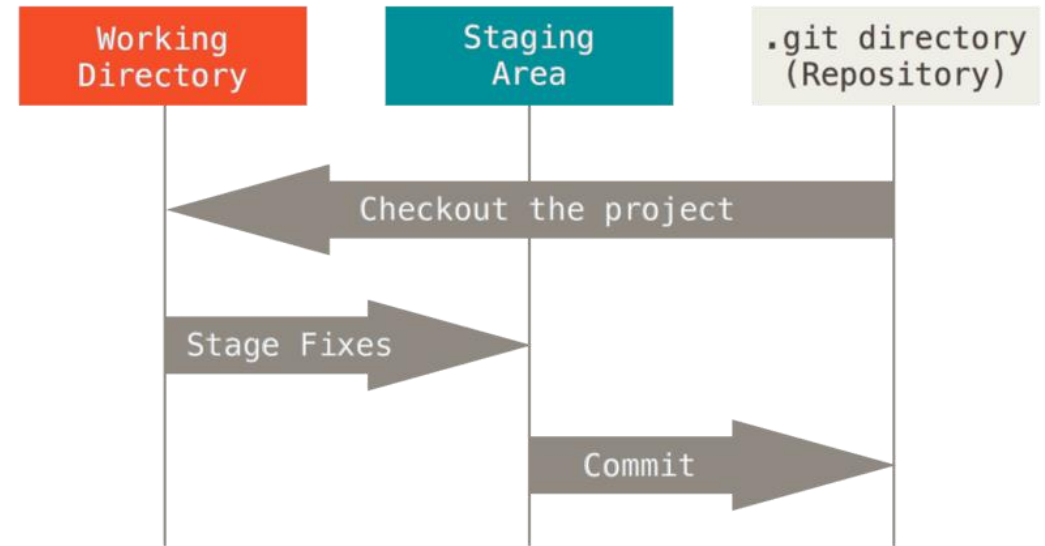
Git Directory

- Where Git stores the metadata and Object database
- Most important part of Git
- This is what is copied when you "clone"



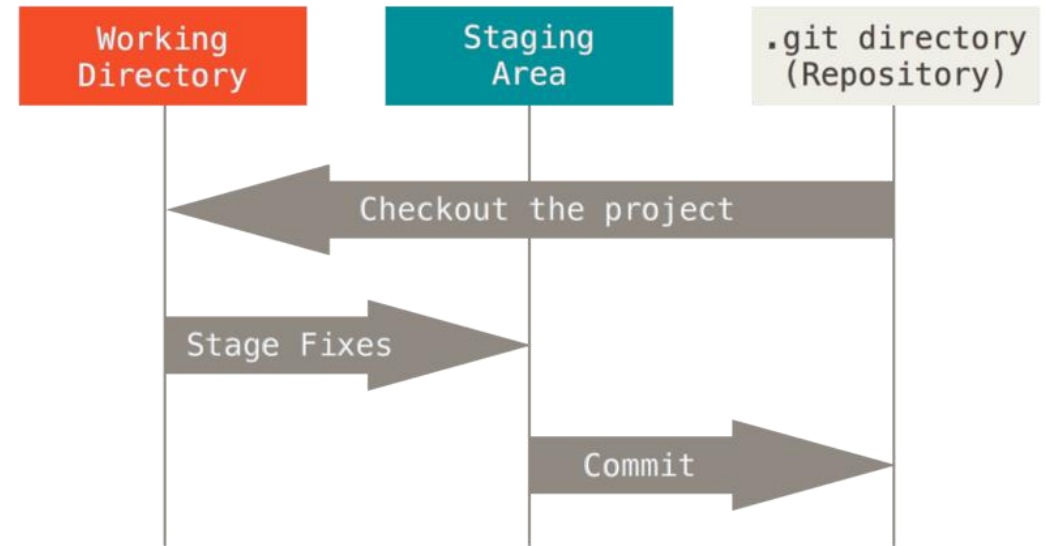
Git Workflow

1. You modify files in your working tree
2. You stage the changes you want to be included in your next commit
3. You do a commit. This takes the files as they are in the staging area and adds them to your .git directory



Git Workflow

“If a particular version of a file is in the Git directory, it’s considered **committed**. If it has been modified and was added to the staging area, it is **staged**. And if it was changed since it was checked out but has not been staged, it is **modified**.”



Git Command Line

- Okay, but how do we actually do this?
- You can use Git from the Command Line
 - But how do we do that?

Git Command Line

- [Installing Git](#)

Git Command Line

Useful Git commands

- `git clone <url>`
- `git checkout <branch>`
- `git pull`
- `git add <files>`
- `git commit -m <"commit message">`
- `git push <branch>`

Git Clone

Useful Git commands

- `git clone <url>`
- `Git clone https://gitlab.com/a4220/mytestrepo.git`

Git Clone

Useful Git commands

- `git clone <url>`
- `Git clone https://gitlab.com/a4220/mytestrepo.git`

GitLab

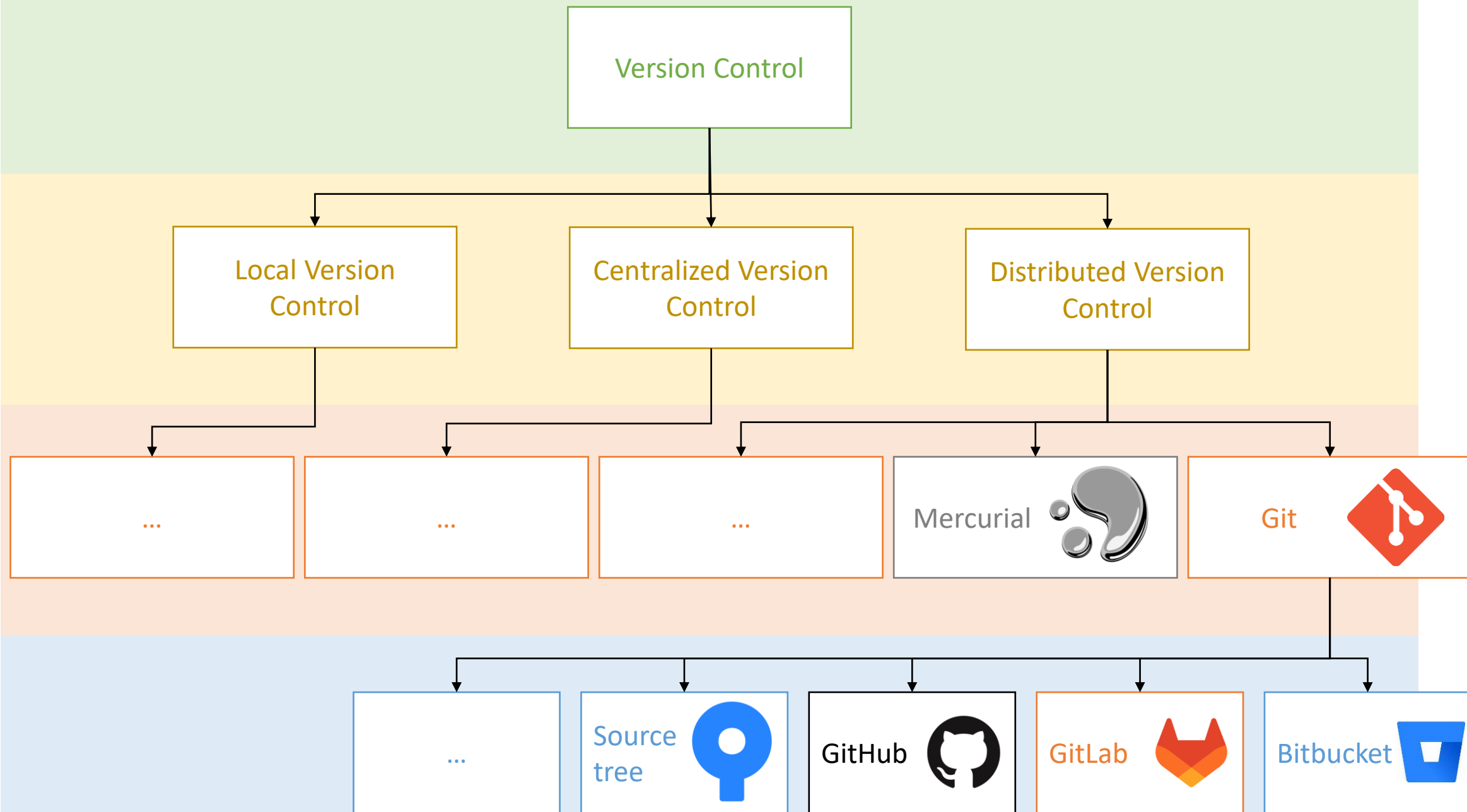
What is GitLab?

- **Git** is a Distributed Version Control System
- **GitLab** is a Git client / Git management software
- Web-based Git management software that provides free public and private repositories

Version Control
Types

Version Control
Systems

Version Control
Clients



GitLab

- Using GitLab we can:
 - Create public and/or private repositories
 - Track issues
 - Handle merge requests
 - And more
- Using Git we can:
 - Clone our GitLab repositories
 - Add, commit, push, pull, branch, merge, and so on.

UoW GitLab

What is UoW GitLab?

- The University has a teaching instance of GitLab (actually a few)
- The SCMS instance is found at courses-git.cms.waikato.ac.nz
- You will need to select Waikato AD and sign in with your CMS (Linux) username and password
- Let's take a look ...

Git and GitLab Example

Git and GitLab Example

Example One: Starting out

1. Create a repository (GitLab)
2. Clone the repository (Git clone)
3. Create a Java file and write some code
4. Add changes to the staging area (Git add)
5. Commit changes to the .git directory (Git commit)
6. Update the online GitLab repository (Git push)

Git and GitLab Example

Example Two: Coming back

- | | |
|---|--|
| 1. Create a repository | (GitLab) |
| 2. Clone the repository | (Git clone) |
| 3. Make sure you have the latest version | (Git fetch && git pull) |
| 4. Create a Java file and write some code | |
| 5. Add changes to the staging area | (Git add) |
| 6. Commit changes to the .git directory | (Git commit) |
| 7. Update the online GitLab repository | (Git push) |

Git and GitLab Example

Example Three: Starting out w Branching

1. Create a repository (GitLab)
2. Create a branch (GitLab)
3. Clone the repository (Git clone)
4. Checkout your branch (Git checkout <branch>)
5. Create a Java file and write some code
6. Add changes to the staging area (Git add)
7. Commit changes to the .git directory (Git commit)
8. Update the online GitLab repository (Git push)

Git and GitLab Example

Example Four: Coming back w Branching

- | | |
|---|--|
| 1. Create a repository | (GitLab) |
| 2. Create a branch | (GitLab) |
| 3. Clone the repository | (Git clone) |
| 4. Checkout your branch | (Git checkout <branch>) |
| 5. Make sure you have the latest version | (Git fetch && git pull) |
| 6. Create a Java file and write some code | |
| 7. Add changes to the staging area | (Git add) |
| 8. Commit changes to the .git directory | (Git commit) |
| 9. Update the online GitLab repository | (Git push) |

Git and GitLab Example

Example Five: Merge your branch with Main

1. Checkout your branch (Git checkout <branch>)
2. Make sure you have the latest version (Git fetch && git pull)
3. Make sure the latest Main is on your branch (Git rebase main)
4. Switch to Main (Git checkout main)
5. Merge your branch with Main (Git merge <branch>)
6. Update the online GitLab repository (Git push)

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Software Development Models

COMPX₃₄₁

Overview

- Engineering development models
- Software Engineering development models
- Waterfall
- Agile

Engineering development models

- A development model is:
 - A way of organising the work of individuals or teams on a project
 - It can involve documents, processes, techniques, and standards
- The use of development models in Software Engineering stem from the use of development models in other fields such as Engineering
- In Engineering, this involves applying science and mathematics to:
 - Find novel solutions
 - Improve existing solutions

Software engineering development models

- Software Engineering lacks the clear and generally agreed processes that other engineering disciplines have developed over long periods of time
- As a result, there is no one software engineering development model. Instead numerous models have been developed over time.

Software development models

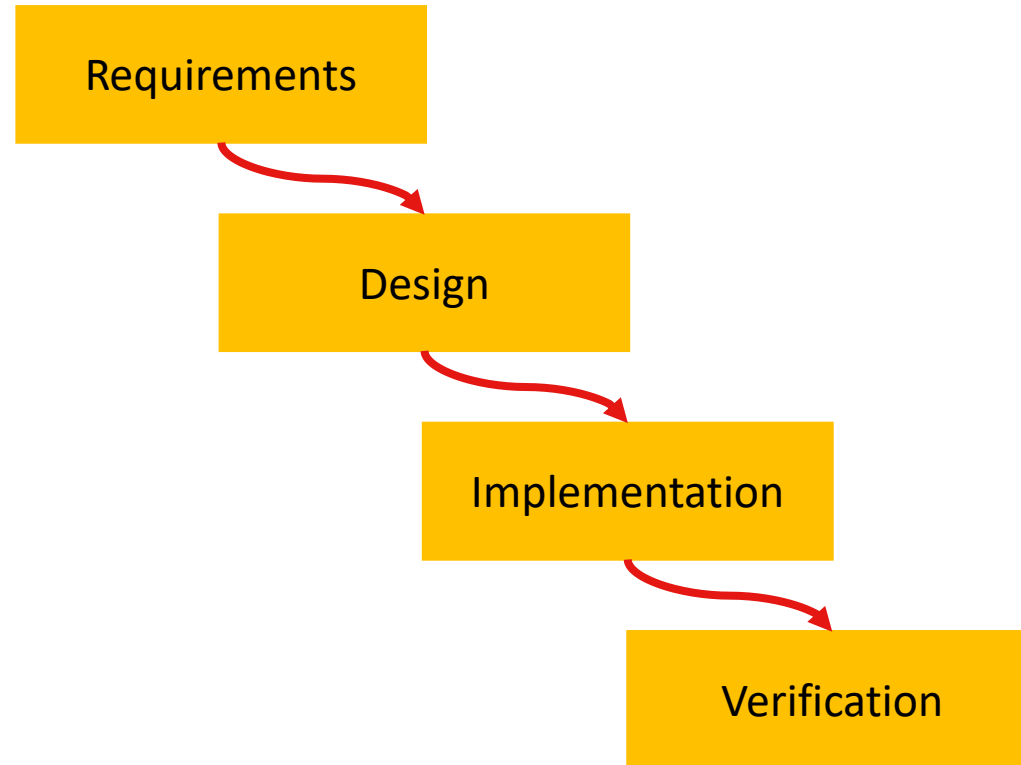
1. Formal specifications / proofs (From Mathematics)
 - Automated checking of programs
2. Rigorous documentation and planning (From other engineering disciplines)
 - Waterfall process
3. Processes (From management and process control)
 - Testing / code review / programming languages / coding standards (DevOps ish)
4. Agile methods (Sort of biological ???)
 - Iterative and incremental development / SCRUM / extreme programming

Waterfall

Waterfall

- Waterfall Software Engineering development model
 - A linear, sequential approach
 - Each phase depends on the deliverables of the previous one

Waterfall



Waterfall

- Originally used in Engineering disciplines
 - Manufacturing and construction industries
- For Software:
 - 1956 – the first waterfall-ish approach was presented (SAGE)
 - 1970 – the first formal description of the waterfall model (Royce)
 - 1976 – the first time the term “waterfall” was used
 - 1985 - the United States Department of Defence adopts waterfall as their standard for working with software development contractors

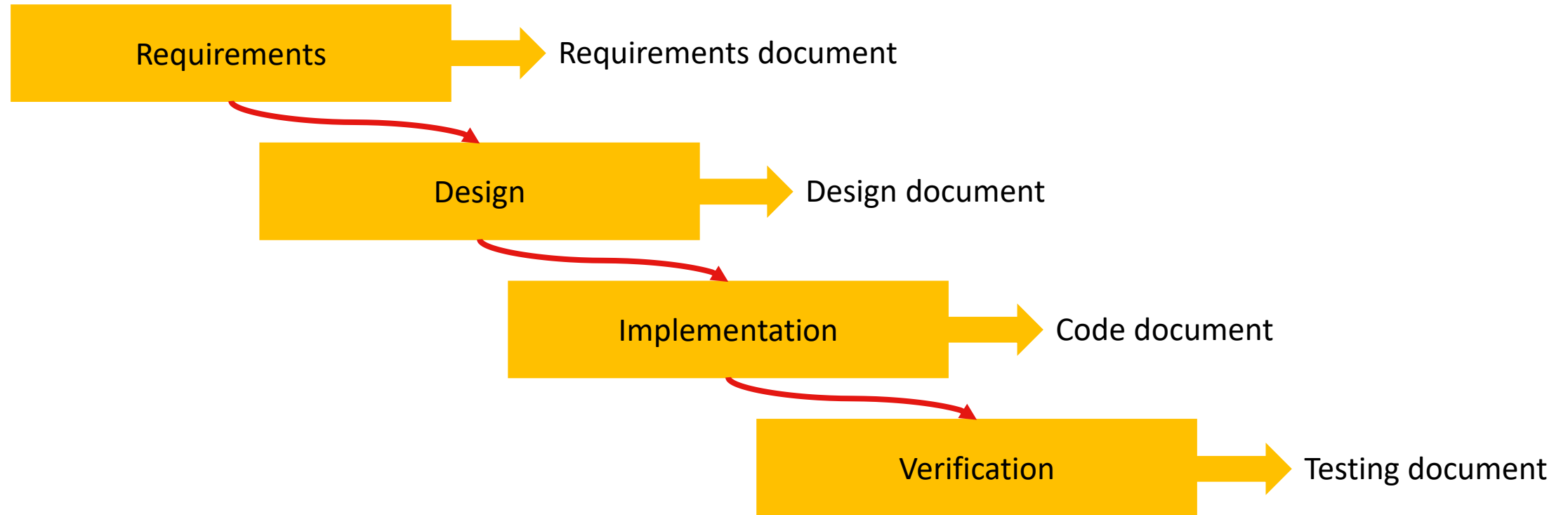
Waterfall

- Fun fact: the 1970 paper that provided the first formal description of the waterfall model was actually using it as an example of a flawed, non-working model.
- Royce 1970 <http://www-scf.usc.edu/~csci201/lectures/Lecture11/royce1970.pdf>

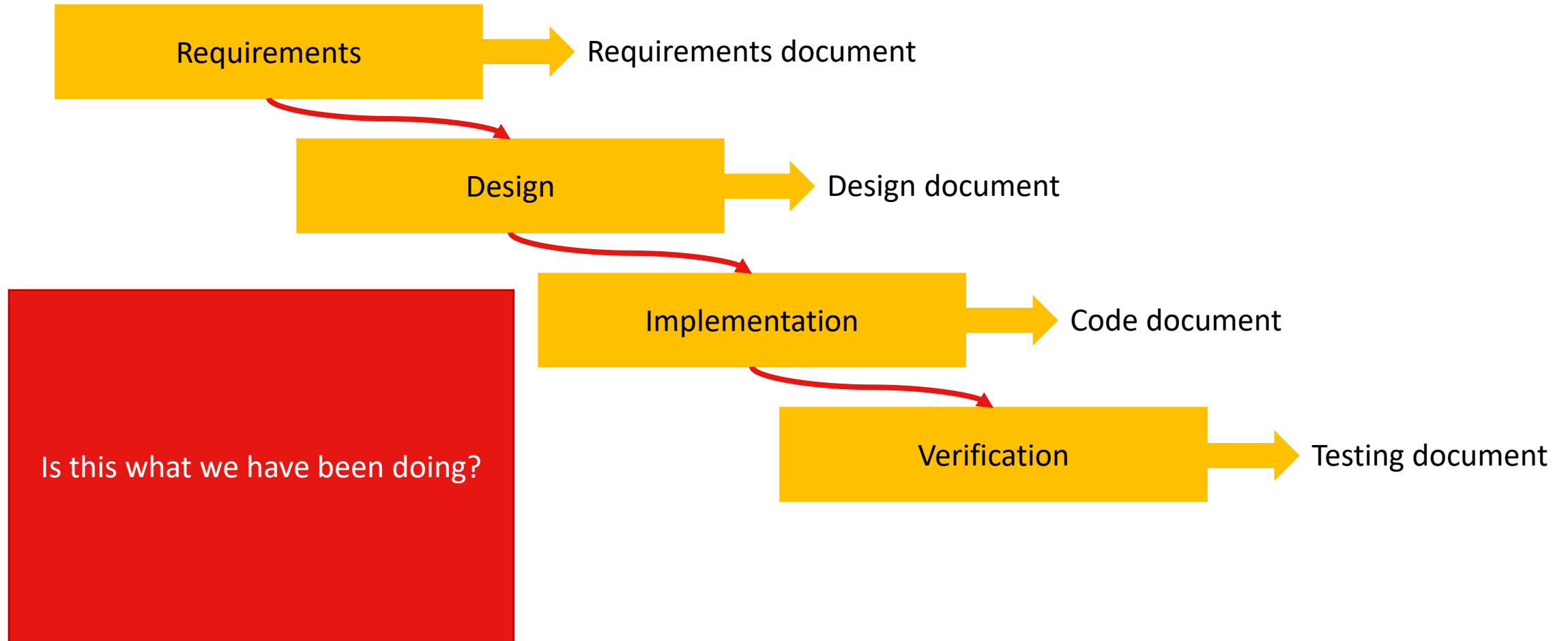
Waterfall

1. Requirements
 - Functional (the features that the application should include)
 - Non functional (hardware specifications, security, etc.)
2. Design
 - Data structures, storage, classes
3. Implementation
 - Coding
4. Verification
 - Testing

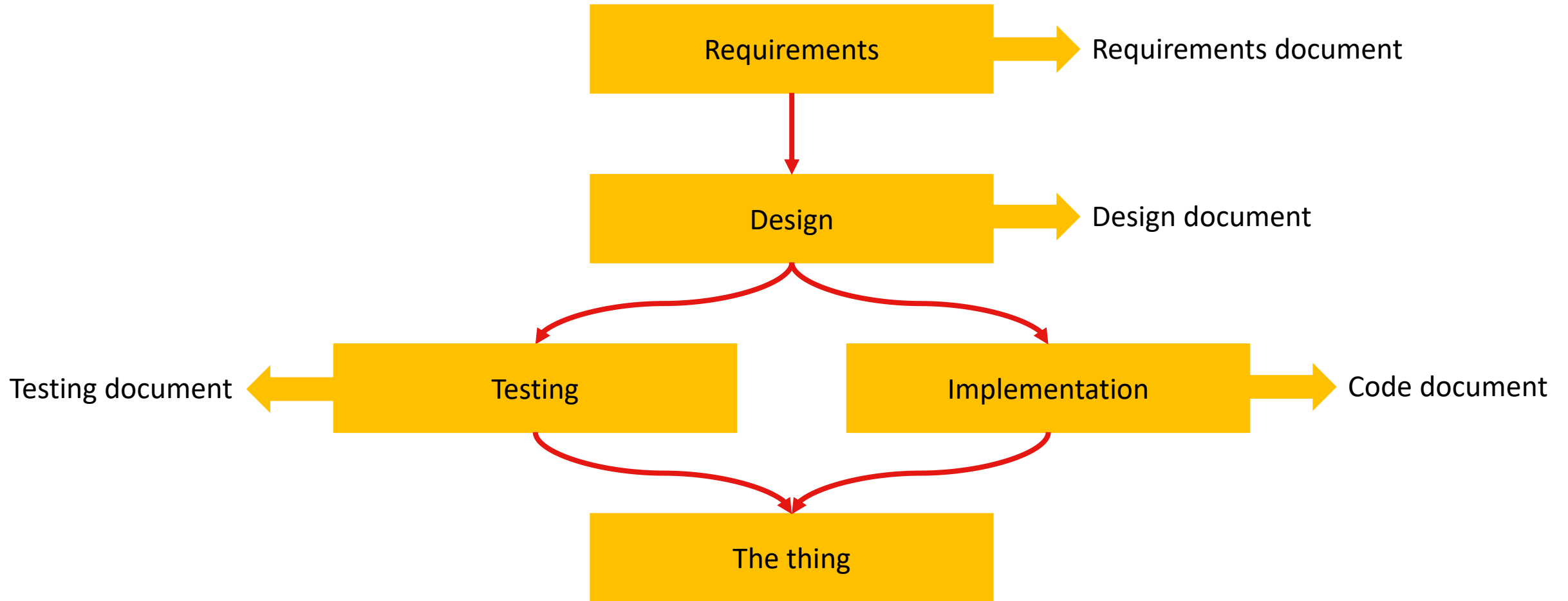
Waterfall



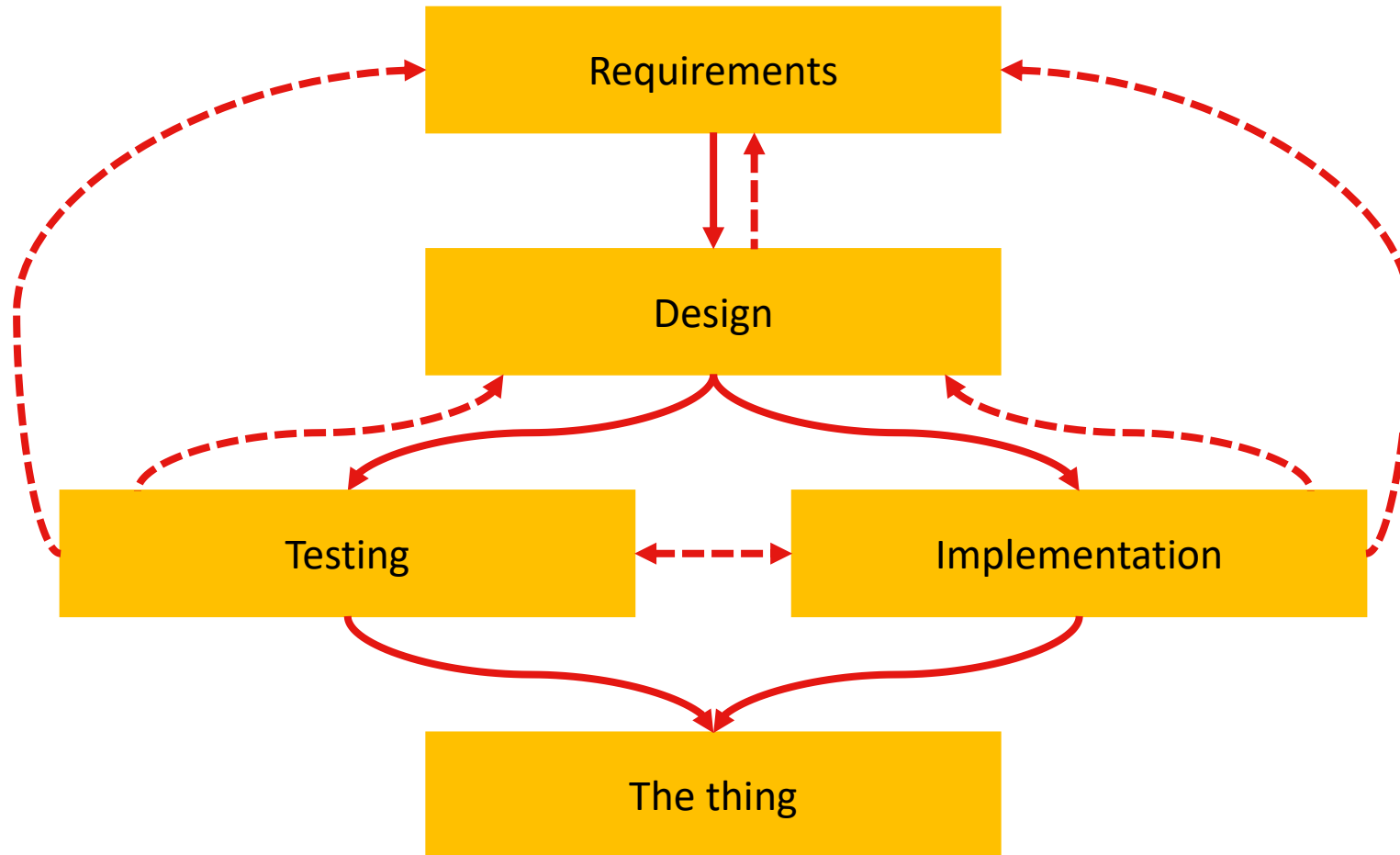
Waterfall



(?)Waterfall



(Not) Waterfall

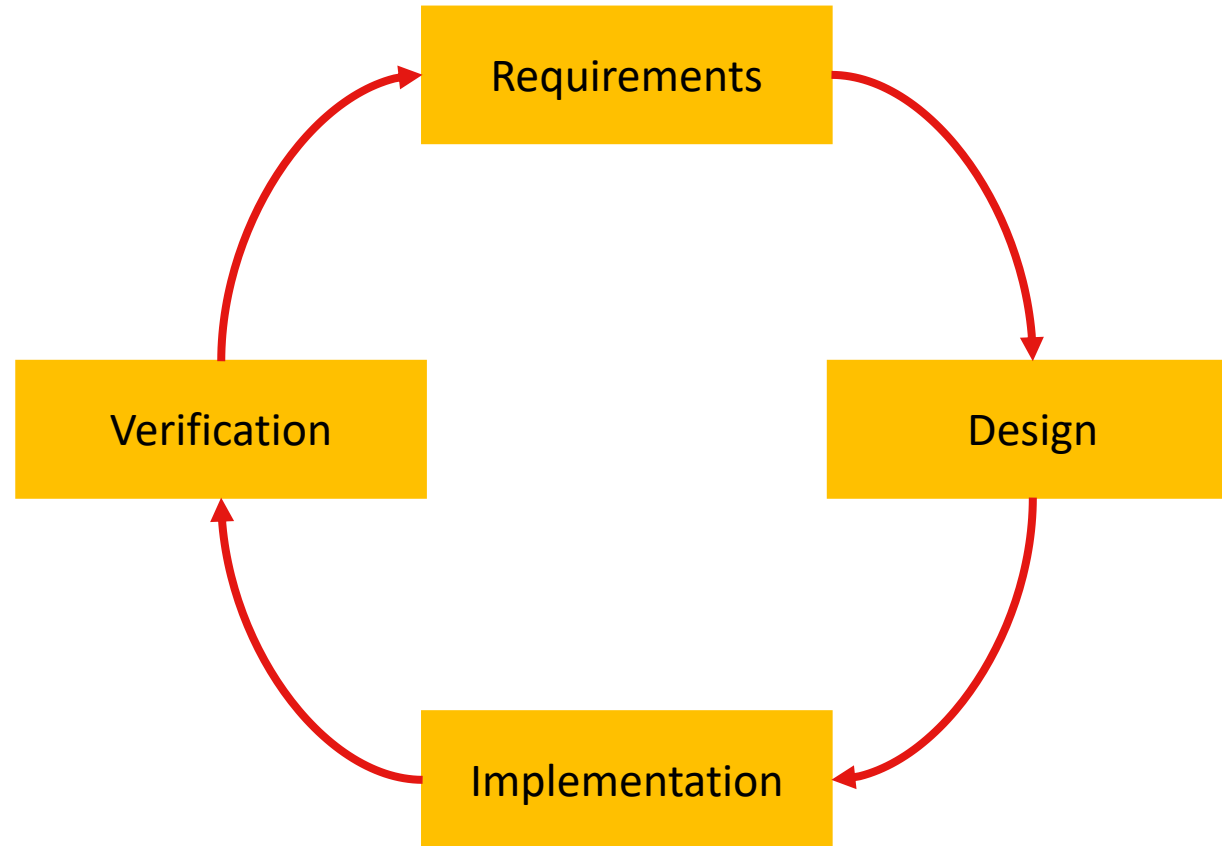


Agile

Agile

- Agile Software Engineering development models
 - Agile in general refers to development models that evolve software rather than require complete upfront design
 - Iterative and incremental

Agile



Agile

- The key observation is that we often can't fully understand a project and its requirements at the start and that we need methods that cope with change

Agile

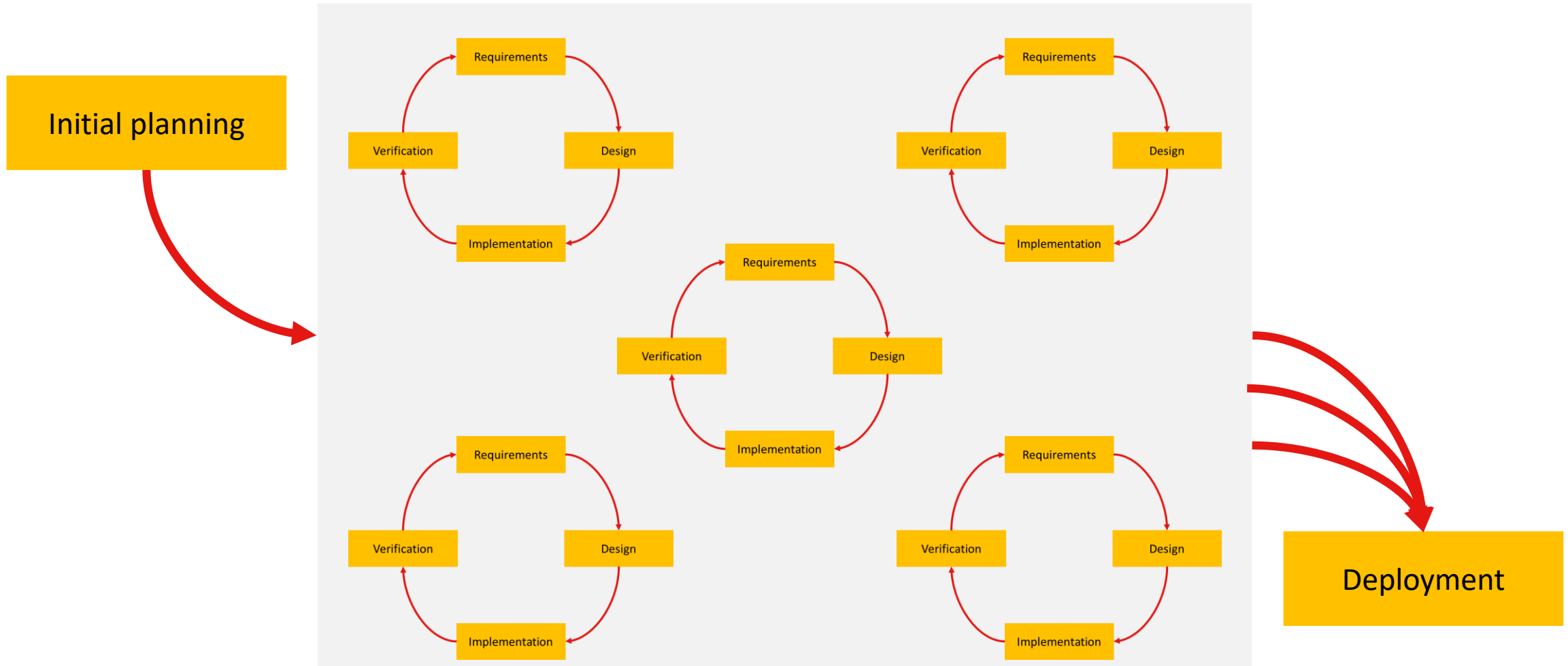
Agile has been around for a long time.

- 1957 – Iterative and incremental development methods (IID) (1 year after WF)
- 1960s – NASA used agile (IID) methods for Project Mercury
- 1970s – adaptive software development
- 1977–1980 – NASA's space shuttle software
- 1990s – a surge in agile methods came about as a “backlash” to waterfall
- 1994 – the United States Department of Defence adopts an agile methodology as their standard for working with software development contractors

Agile

- Breaking a complex task down into smaller, more manageable chunks.
 - Multiple iterations: requirements, design, implementation, and testing
 - Allows for changes throughout the process
 - Smaller chunks can be understood separately

Agile



Agile example

- You have been asked to develop a VERY basic social media platform.

Agile example

1. Initial planning

- Create a preliminary release plan
 - Must haves
 - Register/login/logout, create profile, post text, post images
 - Should haves
 - Post videos, comment on posts
 - Could haves
 - Likes, follows

Agile example

2. Iteration A

- Focused on “registering”
 - Requirements
 - Design
 - Implementation
 - Testing

Agile example

3. Iteration B

- Focused on “login/logout”
 - Requirements
 - Design
 - Implementation
 - Testing

Agile example

4. Iteration C

- Focused on “creating a profile”
 - Requirements
 - Design
 - Implementation
 - Testing

Agile example

- Iterations can be completed simultaneously by different groups of developers
 - Frequent integration
 - Frequent testing
- Finished features can be deployed (released) while other features are still under development.
- If you find a mistake, you do not have to start from the beginning again

Agile example

- This is a “generalized” description of Agile
- There are A LOT of different Agile development models
 - Iterative and incremental
 - Extreme programming
 - SCRUM
 - ...

Iterative and Incremental

Iterative and Incremental

- Develop a system through
 - Repeated cycles (iterative)
 - And in smaller portions (incremental)

Iterative and Incremental

- Three stages:
 - Initialization step
 - Project control list (increments)
 - Iteration step

1. Initialization

The initialization step creates a base version of the system.

- Provides a product to which the user can react
- Offers some of the key aspects
- Must be simple enough to understand and implement easily

2. Project Control List

The project control list breaks the project into increments (tasks)

- Contains a record of all tasks to be performed
- Includes new features
- And redesigns of existing features
- Constantly revised

3. Iteration

- Involves iterating through phases to complete tasks from the project control list
- Each phase can be iterated over 1 or more times (time boxed), revising the project control list as needed

3. Iteration

- Four phases:
 - Inception (scope, requirements, risks)
 - Elaboration (architecture)
 - Construction (production ready code)
 - Transition (release)

3. Iteration

- Four phases:
 - Inception (scope, requirements, risks) (Requirements)
 - Elaboration (architecture) (Design)
 - Construction (production ready code) (Implementation)
 - Transition (release) (Testing and release)

Extreme Programming (XP)

Extreme Programming

- Advocates frequent "releases" in short development cycles
 - reduce the cost of changes in requirements
 - introduce checkpoints where new requirements can be added
 - improve productivity
- Does not define a stable set of requirements
- Changes are natural, inescapable and desirable

Extreme Programming

- Elements of Extreme Programming (XP)
 - extensive code review
 - programming in pairs (peer programming)
 - unit testing of all code
 - avoiding programming of features until they are actually needed
 - expecting changes in the customer's requirements as time passes and the problem is better understood
 - frequent communication with the customer and among programmers

Extreme Programming

- There are four “basic activities” performed when using XP:
 1. Coding
 2. Testing
 3. Listening
 4. Designing

1. Coding

XP doctrine: The only truly important product of the system development process is
CODE

- Use code to
 - Figure out the most suitable solution
 - Communicate thoughts about programming problems
 - Code is always clear and concise and cannot be interpreted in more than one way
 - Other programmers can give feedback by also “coding their thoughts”

1. Coding

- Coding Rules
 - The customer is always available
 - Code the unit test first
 - Only one pair integrates code at a time
 - Leave optimization until last
 - No overtime

2. Testing

- XP's approach is that if a little testing can eliminate a few flaws, a lot of testing can eliminate many more flaws.
 - Unit tests
 - Acceptance tests
 - Integration testing

2. Testing

- Testing Rules:
 - All code must have unit tests
 - All code must pass all unit tests before it can be released.
 - When a bug is found, tests are created before the bug is addressed (a bug is not an error in logic; it is a test that was not written)
 - Acceptance tests are run often and the results are published

3. Listening

Programmers must listen to what the customer needs the system to do and give the customer feedback

- The planning game
 - A meeting that occurs ~ once a week
 - Release planning (customer and developer)
 - Iteration planning (developer)

4. Designing

- XP is centred on coding, testing, and listening
 - Argued that if it is done well, design is not needed
 - Sufficient for smaller projects ...
- BUT
 - in reality, a system becomes too complex, or dependencies become unclear

4. Designing

- Designing involves:
 - Creating a design structure
 - Avoiding lots of dependencies
 - Ensuring that changing one part of a system won't affect other parts

XP Values

- Communication
 - Between customers and programmers
 - As opposed to documentation
- Simplicity
 - Starting with the simplest solution
 - Developing and coding for the needs of today
- Feedback
 - From the system (unit tests)
 - From the customer (acceptance tests)
 - From the team (planning game)
- Courage
 - Knowing when to throw code away
- Respect
 - Your own work (strive for high quality)
 - The work of others (don't commit changes that break existing code)

SCRUM

Scrum

- Scrum is a lightweight, iterative and incremental framework for managing complex work.

Scrum

- Key principles:
 - Customers will change their minds
 - There will be unpredictable changes
 - Accepts that a problem cannot be fully understood or defined up front
 - Quick deliverables
 - Responding to emerging requirements
 - Adapts to evolving technologies

Scrum

- Key Features:
 - Roles
 - Workflow (Sprints)
 - Artefacts (Backlog, Burn down charts, etc.)

Scrum roles

- Product owner
 - Voice of the customer
- Development team
 - 3-9 members
- Scrum master
 - Buffer between the team and distracting influences

Scrum workflow

- Sprint
 - Time-boxed development
 - Between 1 week and 1 month

Sprint meetings

- Sprint planning
 - ~ 4 hours (for a 2 week sprint), at the start of a sprint
 - Select items from the product backlog
 - Define sprint backlog
- Daily scrum
 - Same time and place every day
 - What did I complete yesterday?
 - What do I plan to complete today?
 - Any impediments?

Sprint meetings

- Sprint review
 - ~ 2 hours (for a two week sprint), at the end of a sprint
 - Review completed work
 - Present work to stakeholders and decide what's next
 - Incomplete work is NOT presented
- Sprint retrospective
 - ~ 1.5 hrs (for a two week sprint), at the end of the sprint
 - What went well during the sprint?
 - What did not go well?
 - What could be improved?

Scrum artifacts

- Product backlog
- Sprint backlog
- Sprint burn-down chart
- Release burn-up chart

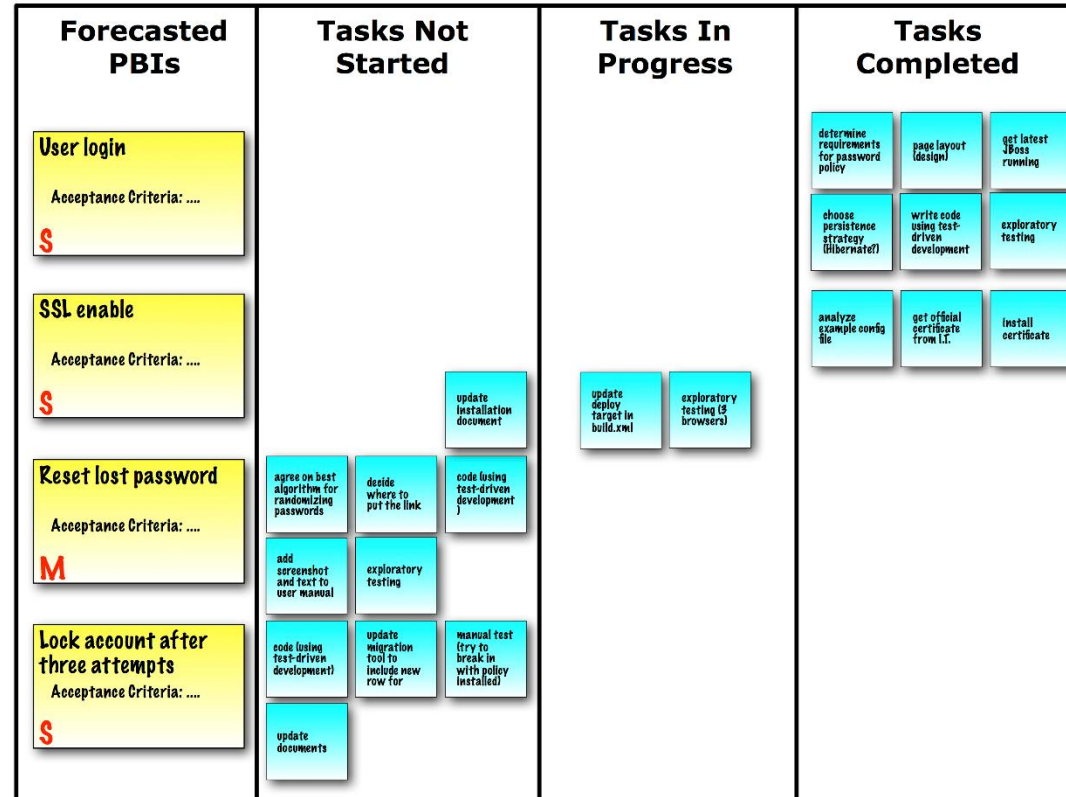
Product backlog

- Product backlog
 - A breakdown of the work that needs to be done
 - Product requirements
 - I.e. user stories, use cases
 - Define features, bug fixes, non-functional requirements
 - Controlled by the “product owner”

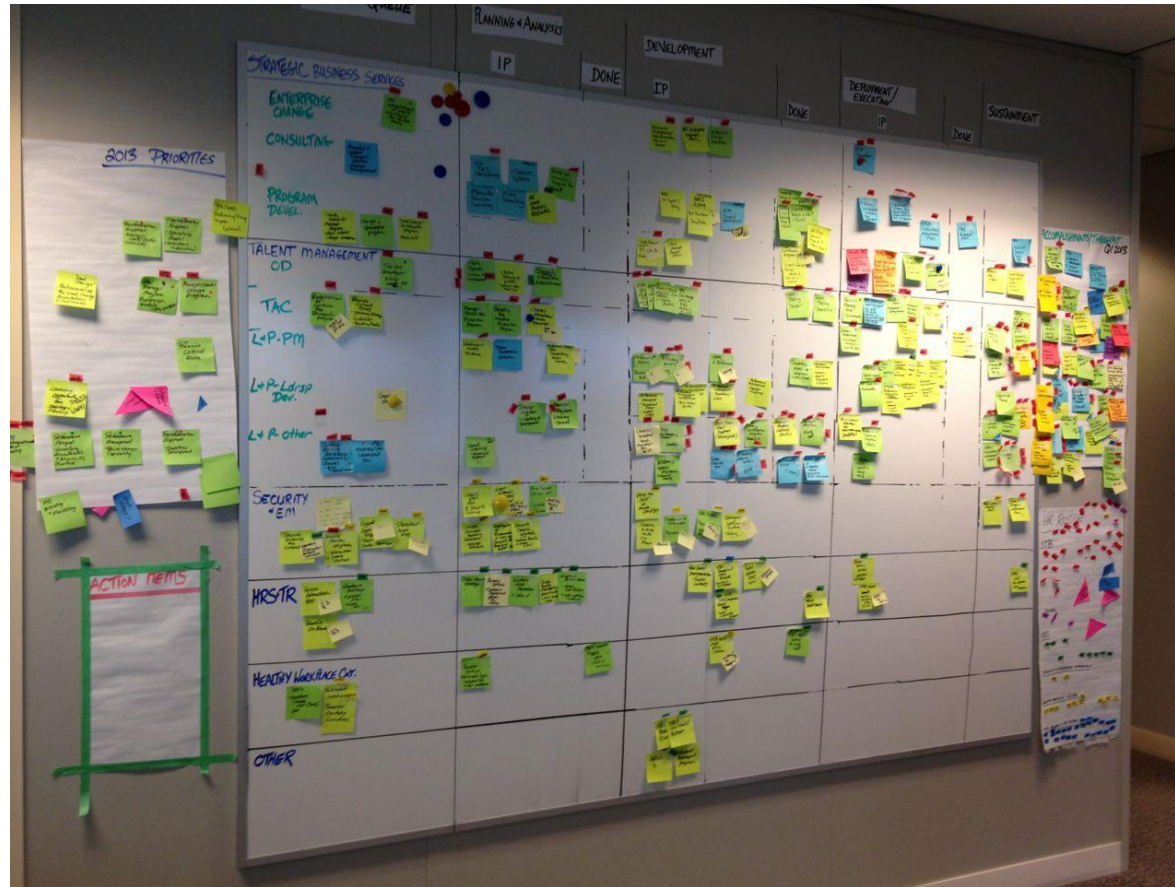
Sprint backlog

- Sprint backlog
 - The work to be addressed during the next sprint
 - Derived from items in the product backlog
 - Broken down into tasks
 - Team members “sign up” to complete a task
 - The full sprint backlog is not always completed in one sprint

Sprint backlog (ideal)



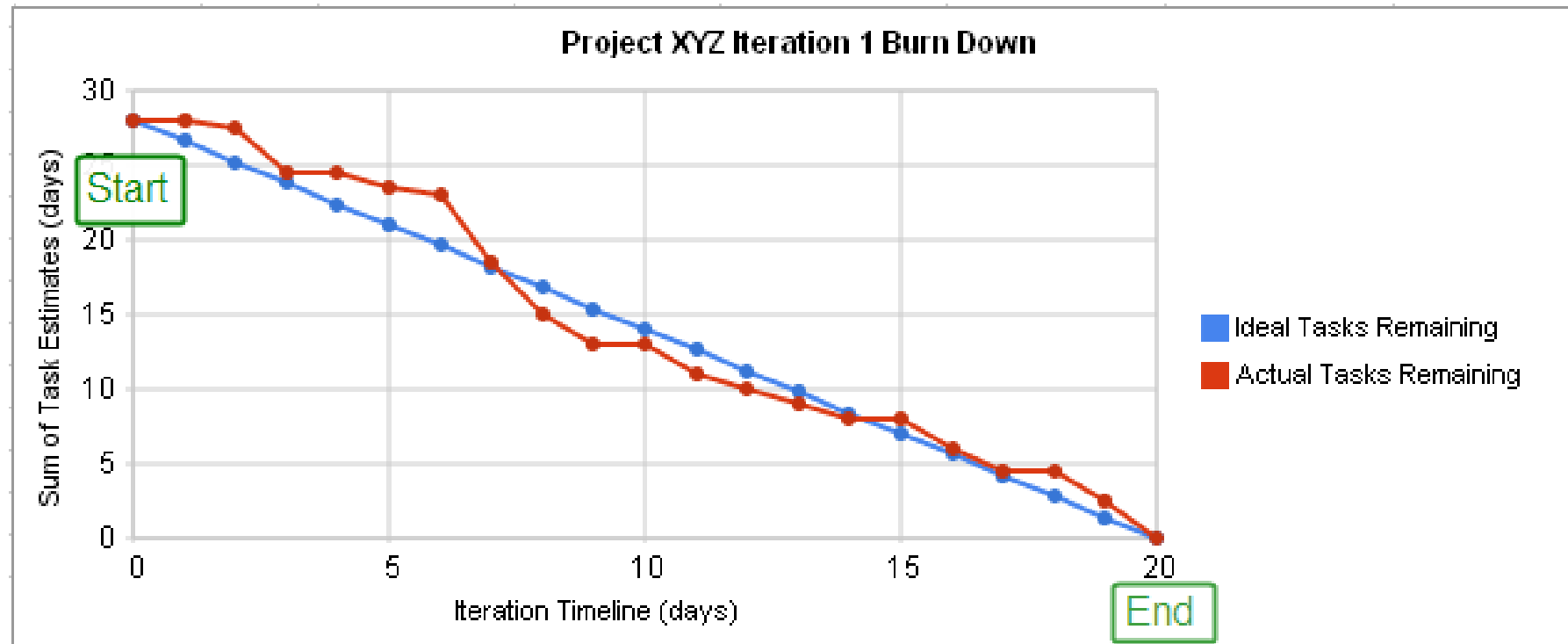
Sprint backlog (real)



Sprint burn-down chart

- Sprint burn-down chart
 - Portrays the work remaining
 - Publically displayed
 - Updated every day
 - “Ideal” is plotted during sprint planning

Sprint burn-down chart (ideal)



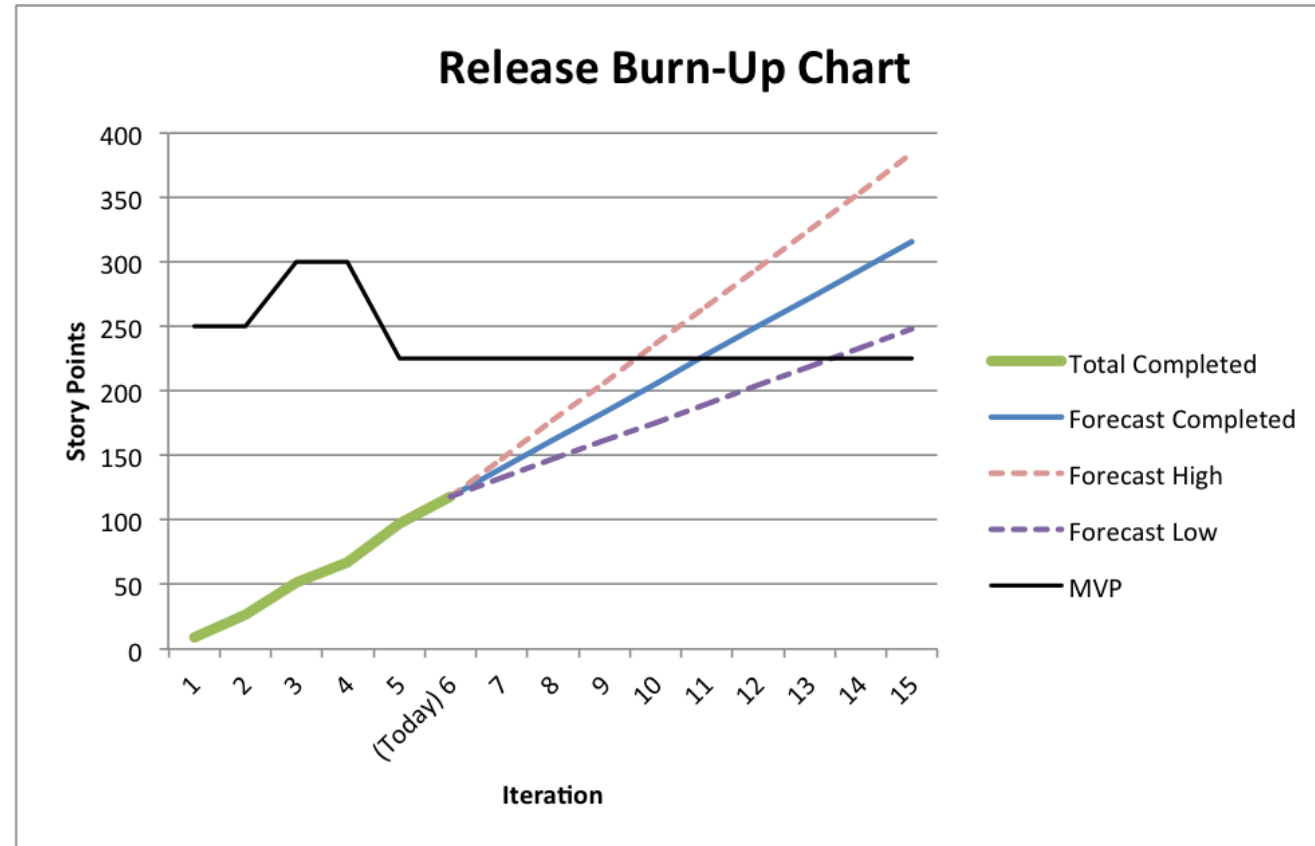
Sprint burn-down chart (real)



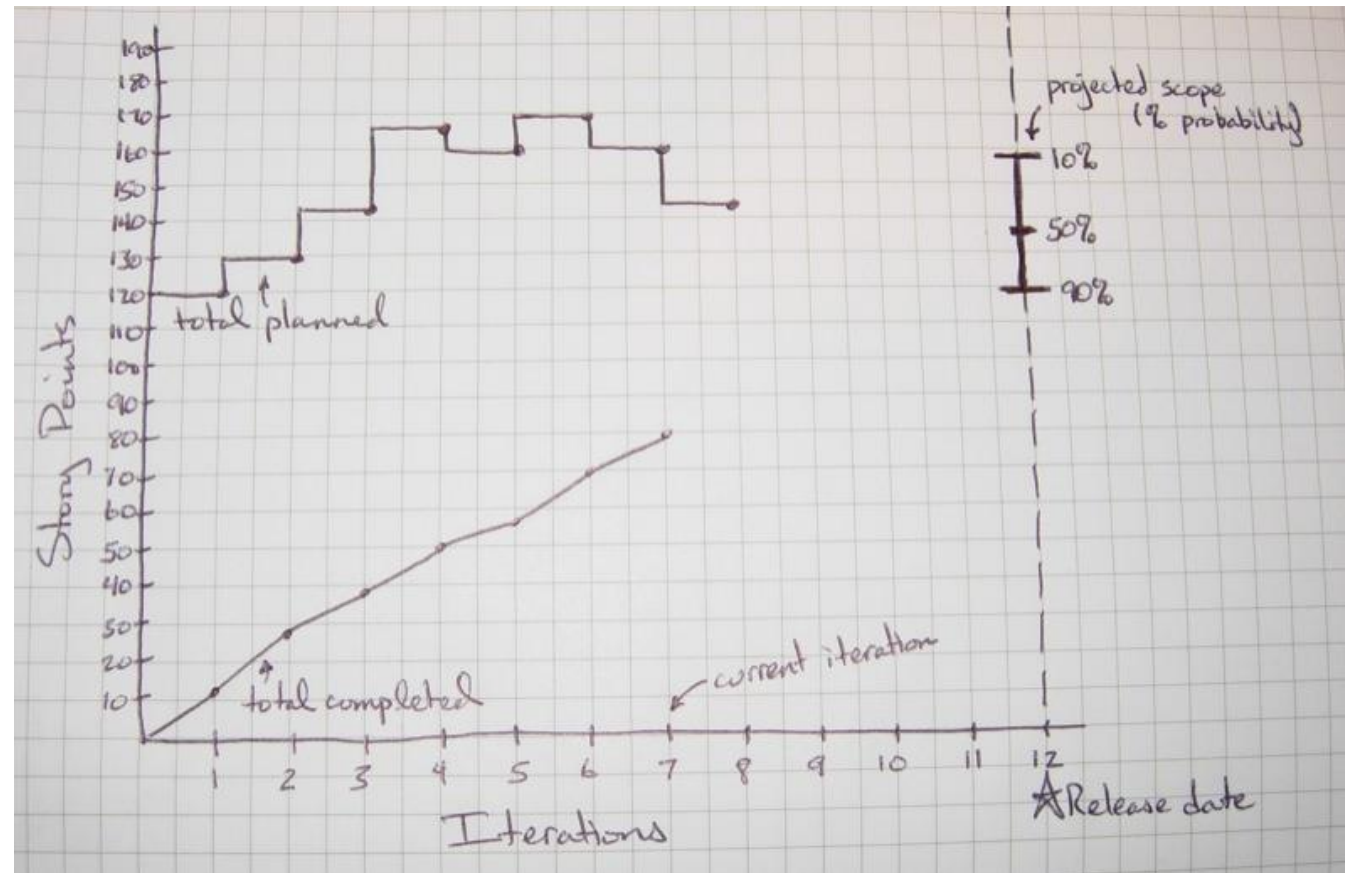
Release burn-up chart

- Release burn-up chart
 - Tracks progress towards a release
 - Updated at the end of each sprint
 - Shows forecasted progress
 - Shows actual progress
 - Shows total work completed/added/removed

Release burn-up chart (ideal)



Release burn-up chart (real-ish)



So, how do we choose?

So .. How do we choose?

- How flexible are your specifications?
- What is your scale and scope?
- Where is your team located?
- How much feedback do you need?
- What is the cost associated with a delay?
- How much change might be requested?

So .. How do we choose?

- The answer ... there is no one right answer
- The software development model that is chosen depends on the project itself, the people running the project, the cost associated with using a particular model, and so on.
- Companies often don't use one development model – they may adopt aspects from several, or use some (but not all) aspects from one
- In fact, companies exist who provide and customise certain development models to suit the needs of their clients (out of the box vs customisation)

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

PRESS RECORD

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Solution Quality

COMPX₃₄₁

Before we start ...

- Testing Standard Out
- Functional Software Test Plan Example

Testing Standard Out

Testing Standard Out: setup

```
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.Assertions.*;
import java.io.*;

public class TestClass {

    private final PrintStream standardOut = System.out;
    private final ByteArrayOutputStream outputStreamCaptor = new ByteArrayOutputStream();

    @BeforeEach
    public void setOut() {
        System.setOut(new PrintStream(outputStreamCaptor));
    }

    // Add tests here

}
```

Testing Standard Out: tests

```
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.Assertions.*;
import java.io.*;

public class TestClass {
    ...
    // Add tests here
    @Test
    @DisplayName("Test Print Method")
    public void testPrint() {
        SomeClass sc = new SomeClass();
        sc.printSomething();
        String actual = outputStreamCaptor.toString().trim();
        Assertions.assertEquals("Some text", actual);
    }
}
```


Testing Standard Out

- Let's give it a try ...

Functional Software Test Plan Example

Functional Software Test Plan Example

- Let's take a look ...

Solution Quality

COMPX₃₄₁

Overview

- Solution quality – why does it matter?
- Minimizing Complexity
- Anticipating Change
- Construction for Verification
- Reuse
- Standards in Construction

Solution Quality – why does it matter?

Solution quality isn't about bugs, it's about structuring your software well. Badly structured software may slow down development or increase the risk of introducing bugs.

Remember from the design content:

- Decomposition/modularisation

- Encapsulation/information hiding

- Sufficiency, completeness, and primitiveness

- Separation of concerns

- Separation of interface and implementation

Solution Quality - fundamentals

1. Minimizing Complexity
2. Anticipating Change
3. Construction for Verification
4. Reuse
5. Standards in Construction

1. Minimizing complexity

- Code creation that is simple and readable, rather than clever
- We can do this by making use of:
 - Standards in Construction
 - Modular Design
 - Coding techniques
 - Code quality

Minimizing complexity (Example)

"We would like to create a spell checker application. The application will take a word and either confirm that it is spelt correctly, or confirm that it is not spelt correctly. Input words may include punctuation, but cannot include numerical characters"

Minimizing complexity (Example)

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word.toLowerCase();
wordCleaned = wordCleaned.replaceAll("[\\p{Punct}\\p{IsPunctuation}]", "");

// Check if word contains a number
Boolean hasNumerical = false;
for(int i = 0; i < wordCleaned.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(wordCleaned.charAt(i)))){
        hasNumerical = true;
    }
}

// Check if it is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = false;
    for(int i = 0; i < dictionary.length; i++){
        if(dictionary[i].equals(wordCleaned)){
            isWord = true;
        }
    }

    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word;
wordCleaned = convertToLower(wordCleaned);
wordCleaned = stripPunctuation(wordCleaned);
Boolean hasNumerical = includesNumerical(wordCleaned);

// Determining whether the word is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = existsInDictionary(wordCleaned);
    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

Minimizing complexity (Reading)

- <https://www.techtarget.com/searchsoftwarequality/tip/How-to-prevent-accidental-complexity-in-software-development>

2. Anticipating change

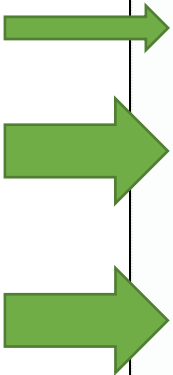
- Most Software will change over time
 - More features added
 - Maintenance performed
 - Change of environment
 - ...

Anticipating change (Example)

"We would like to create a spell checker application. The application will take a word and either confirm that it is spelt correctly, or confirm that it is not spelt correctly. Input words may include punctuation, but cannot include numerical characters"

" added feature, the spell checker should be able to accept words that include numerical characters – these characters will just be stripped from the word"*

Anticipating change (Example)



```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word.toLowerCase();
wordCleaned = wordCleaned.replaceAll("[\\p{Punct}\\p{IsPunctuation}]", "");

// Check if word contains a number
Boolean hasNumerical = false;
for(int i = 0; i < wordCleaned.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(wordCleaned.charAt(i)))){
        hasNumerical = true;
    }
}

// Check if it is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = false;
    for(int i = 0; i < dictionary.length; i++){
        if(dictionary[i].equals(wordCleaned)){
            isWord = true;
        }
    }

    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

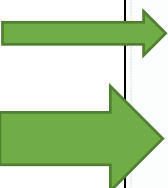
// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word.toLowerCase();
wordCleaned = wordCleaned.replaceAll("[\\p{Punct}\\p{IsPunctuation}]", "");
wordCleaned = wordCleaned.replaceAll("[0-9]", "");

Boolean isWord = false;
for(int i = 0; i < dictionary.length; i++){
    if(dictionary[i].equals(wordCleaned)){
        isWord = true;
    }
}

if(isWord){
    System.out.println("It's a word!");
} else {
    System.out.println("It's NOT a word!");
}
```

Anticipating change (Example)



```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word;
wordCleaned = convertToLower(wordCleaned);
wordCleaned = stripPunctuation(wordCleaned);
Boolean hasNumerical = includesNumerical(wordCleaned);

// Determining whether the word is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = existsInDictionary(wordCleaned);
    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word;
wordCleaned = convertToLower(wordCleaned);
wordCleaned = stripPunctuation(wordCleaned);
wordCleaned = stripNumerals(wordCleaned);

// Determining whether the word is a 'real' word
Boolean isWord = existsInDictionary(wordCleaned);
if(isWord){
    System.out.println("It's a word!");
} else {
    System.out.println("It's NOT a word!");
}
```

3. Constructing for Verification

- When you are building the software, think about how it will be tested.
- We can do this by making use of:
 - Coding standards
 - Code reviews and unit testing
 - Organizing code to support automated testing
 - Restricting the use of complex or hard-to-understand language structures
 - ...

Constructing for Verification (Example)

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word.toLowerCase();
wordCleaned = wordCleaned.replaceAll("[\\p{Punct}\\p{IsPunctuation}]", "");

// Check if word contains a number
Boolean hasNumerical = false;
for(int i = 0; i < wordCleaned.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(wordCleaned.charAt(i)))){
        hasNumerical = true;
    }
}

// Check if it is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = false;
    for(int i = 0; i < dictionary.length; i++){
        if(dictionary[i].equals(wordCleaned)){
            isWord = true;
        }
    }

    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

- Unit tests?
- Main method

Constructing for Verification (Example)

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word;
wordCleaned = convertToLower(wordCleaned);
wordCleaned = stripPunctuation(wordCleaned);
Boolean hasNumerical = includesNumerical(wordCleaned);

// Determining whether the word is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = existsInDictionary(wordCleaned);
    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

- Unit tests?
 - convertToLower()
 - stripPunctuation()
 - includesNumerical()
 - existsInDictionary()
 - Main method

4. Reuse

- Using existing assets to solve different problems
- Assets include:
 - Libraries
 - Modules
 - Components
 - Source code
 - Commercial off-the-shelf assets
- Note “construction *for* reuse” versus “construction *with* reuse”

Reuse (Example)

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word.toLowerCase();
wordCleaned = wordCleaned.replaceAll("[\\p{Punct}\\p{IsPunctuation}]", "");

// Check if word contains a number
Boolean hasNumerical = false;
for(int i = 0; i < wordCleaned.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(wordCleaned.charAt(i)))){
        hasNumerical = true;
    }
}

// Check if it is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = false;
    for(int i = 0; i < dictionary.length; i++){
        if(dictionary[i].equals(wordCleaned)){
            isWord = true;
        }
    }

    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

- What can we reuse?
 - Overall method

Reuse (Example)

```
// Prompting a user to enter a word (or sequence of characters)
System.out.println("Please type a word:");

// Using Scanner for Getting Input from User
Scanner in = new Scanner(System.in);
String word = in.nextLine();

// Clean the word (lower case and remove punctuation)
String wordCleaned = word;
wordCleaned = convertToLower(wordCleaned);
wordCleaned = stripPunctuation(wordCleaned);
Boolean hasNumerical = includesNumerical(wordCleaned);

// Determining whether the word is a 'real' word
if(hasNumerical){
    System.out.println("The word cannot include numerical characters.");
} else{
    Boolean isWord = existsInDictionary(wordCleaned);
    if(isWord){
        System.out.println("It's a word!");
    } else {
        System.out.println("It's NOT a word!");
    }
}
```

- What can we reuse?
 - convertToLower()
 - stripPunctuation()
 - includesNumerical()
 - existsInDictionary()
 - Overall method

5. Standards in Construction

Standards can include:

- Communication methods (e.g. document formats and contents)
- Programming languages (e.g. language standards)
- Coding standards (e.g. naming conventions etc.) – more about this next time!
- Platforms (e.g. interface standards for system calls)
- Tools (e.g. diagrammatic standards like UML)

Also, note, external versus internal standards.

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Code Quality

COMPX₃₄₁

Before we start

- Coursework 2 (preliminary) feedback
- Quotation marks in LaTeX
- Coursework 3
 - Let's take a look ...

Code Quality

COMPX₃₄₁

Overview

- Code quality – why does it matter?
- Formatting
- Naming conventions
- Comments
- Refactoring

Code quality – why does it matter?

Code quality isn't about bugs, it's about writing *good* code. Badly written code may slow down development or increase the risk of introducing bugs.

Good code should be tidy, include comments, and use universal conventions that make it easier to read and understand.

Code quality

There's a lot! But we will look at the following:

- Formatting
 - Indentation
 - Whitespace
 - Brackets
- Naming conventions
 - Class names
 - Method names
 - Variable names
- Commenting
 - Class level
 - Method level
 - Above line and inline
 - Java docs
- Refactoring

Formatting

Formatting: Indentation

In Java, indentation doesn't have to be correct for your code to run, but it does make it a lot easier to read! ...

```
boolean dictinoaryHasWord = false;
for(int i = 0; i < dictionary.length; i++){
    if(dictionary[i].equals(word)){
        dictinoaryHasWord = true;
    }
}

if(dictinoaryHasWord){
    System.out.println(testWord + " is a real word!");
} else {
    System.out.println(testWord + " is not a real word");
}
```

```
boolean dictinoaryHasWord = false;
for(int i = 0; i < dictionary.length; i++){
    if(dictionary[i].equals(word)){
        dictinoaryHasWord = true;
    }
}

if(dictinoaryHasWord){
    System.out.println(testWord + " is a real word!");
} else {
    System.out.println(testWord + " is not a real word");
}
```


Formatting: Whitespace

In Java, whitespace doesn't have to be correct for your code to run, but it does make it a lot easier to read! ...

```
String testWord = "time";
String word = testWord;
boolean includesNumerical = false;
for (int i = 0; i < word.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
        includesNumerical = true;
    }
}
if(includesNumerical){

    System.out.println("Sorry, there are numbers");
}
```

```
String testWord = "time";
String word = testWord;

boolean includesNumerical = false;

for (int i = 0; i < word.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
        includesNumerical = true;
    }
}

if(includesNumerical){
    System.out.println("Sorry, there are numbers");
}
```

Formatting: Whitespace

Whitespace is not just about newlines, but also about spaces within a line of code

```
String testWord="time";
String word=testWord;
boolean includesNumerical=false;
for (int i=0;i<word.length();i++){
    if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
        includesNumerical=true;
    }
}
if(includesNumerical){
    System.out.println("S
}
```

```
String testWord = "time";
String word = testWord;
boolean includesNumerical = false;
for (int i = 0; i < word.length(); i++){
    if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
        includesNumerical = true;
    }
}
System.out.println("S
    e numbers");
```

Note, spaces CAN actually introduce bugs in your code!

l.e. `current != null`
versus `current! =null`

Formatting: Brackets

In Java, the opening bracket should be on the same line as the statement. The closing bracket should be on a new line.

```
else {
    word = word.toLowerCase();
    word = word.replaceAll("[.!?\\-]", "");

    boolean dictionaryHasWord = false;
    for(int i = 0; i < dictionary.length; i++){
        if(dictionary[i].equals(word)){
            dictionaryHasWord = true; } }

    if(dictionaryHasWord){
        System.out.println(testWord + " is a real word!"); } else {
        System.out.println(testWord + " is not a real word"); } }
```

```
else {
    word = word.toLowerCase();
    word = word.replaceAll("[.!?\\-]", "");

    boolean dictionaryHasWord = false;
    for(int i = 0; i < dictionary.length; i++){
        if(dictionary[i].equals(word)){
            dictionaryHasWord = true;
        }
    }

    if(dictionaryHasWord){
        System.out.println(testWord + " is a real word!");
    } else {
        System.out.println(testWord + " is not a real word");
    }
}
```

Naming conventions

Naming conventions (for Java)

- In general names should be:

- Class names should be TitleCase.
- Method names should be camelCase.
- Variable names should be camelCase.

i.e. `public class SpellChecker`

i.e. `public String convertToLower()`

i.e. `String testWord = "time";`

- Note, this applies to both single and compound-word names:

- Class names should be TitleCase.
- Method names should be camelCase.
- Variable names should be camelCase.

i.e. `public class Main`

i.e. `public void main()`

i.e. `String word = testWord;`

Naming conventions (for Java)

- Note, these naming conventions are for Java. Other languages may use different naming conventions.
- I.e. python often uses `under_scores` rather than `camelCase`

Naming conventions (for Java)

- Also, in general:

- Class names should be nouns. i.e. `public class Dictionary`
- Method names should be verbs (actions). i.e. `public void convertToLower()`
- Variable names should be nouns. i.e. `String word = "cat";`
- Except Booleans which should be questions. i.e. `boolean includesNumerals = false;`

Naming conventions (for Java)

- Names should be descriptive

i.e. `int s = 0;`

versus `int smallest = 0;`

- But not overly long or complex

i.e. `int theSmallestNumber = 0;`

Note, this applies to classes, methods, and variables

Commenting

Commenting

- You should comment your code at:
 - Class level
 - Method level
 - Above line – for single lines or blocks of code
 - Inline ONLY if that line is complex

Commenting

```
/**
 * This class contains all of the logic
 * to check if a word is spelled correctly
 * (i.e. exists in the dictionary provided)
 *
 * @author Jemma König
 * @version 0.0.1
 * @since 2023-05-08
 */
public class Main {

    // The dictionary of all English words
    private static String[] dictionary = {"time", "year", "people"}

    /**
     * The main entry point for this class
     * @param args. A string array of arguments (not used)
     */
    public static void main(String[] args){

        String testWord = "Time";
        String word = testWord;

        // Check whether the word contains numbers
        boolean includesNumerical = false;
        for (int i = 0; i < word.length(); i++){
            if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
                includesNumerical = true;
            }
        }

        // If the word contains numbers, end the program here
    }
}
```

Commenting

Class level

Above line
(not inline)

Method level

```
/**
 * This class contains all of the logic
 * to check if a word is spelled correctly
 * (i.e. exists in the dictionary provided)
 *
 * @author Jemma König
 * @version 0.0.1
 * @since 2023-05-08
 */
public class Main {

    // The dictionary of all English words
    private static String[] dictionary = {"time", "year", "people"}

    /**
     * The main entry point for this class
     * @param args. A string array of arguments (not used)
     */
    public static void main(String[] args){

        String testWord = "Time";
        String word = testWord;

        // Check whether the word contains numbers
        boolean includesNumerical = false;
        for (int i = 0; i < word.length(); i++){
            if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
                includesNumerical = true;
            }
        }

        // If the word contains numbers, end the program here
    }
}
```

Commenting

```
/**
 * The main entry point for this class
 * @param args. A string array of arguments (not used)
 */
public static void main(String[] args){

    String testWord = "Time";
    String word = testWord;

    // Check whether the word contains numbers
    boolean includesNumerical = false;
    for (int i = 0; i < word.length(); i++){
        if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
            includesNumerical = true;
        }
    }

    // If the word contains numbers, end the program here
    // and produce an error message for the user
    if(includesNumerical){
        System.out.println("Sorry, there are numbers");
    }

    // If the word doesn't contain numbers, then continue
    else {

        // Convert to lower case and strip punctuation
        word = word.toLowerCase();
        word = word.replaceAll("[.!?\\"-]", "");

        // Check if the word exists in the dictionary
        boolean dictionaryHasWord = false;
        for(int i = 0; i < dictionary.length; i++){
```

Commenting

Method level

Above line
(for blocks of
code)

Above line
(for blocks of
code)

Above line
(for blocks of
code)

Above line
(for clarification)

```
/**
 * The main entry point for this class
 * @param args. A string array of arguments (not used)
 */
public static void main(String[] args){

    String testWord = "Time";
    String word = testWord;

    // Check whether the word contains numbers
    boolean includesNumerical = false;
    for (int i = 0; i < word.length(); i++){
        if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
            includesNumerical = true;
        }
    }

    // If the word contains numbers, end the program here
    // and produce an error message for the user
    if(includesNumerical){
        System.out.println("Sorry, there are numbers");
    }

    // If the word doesn't contain numbers, then continue
    else {

        // Convert to lower case and strip punctuation
        word = word.toLowerCase();
        word = word.replaceAll("[.!?\\-]", "");

        // Check if the word exists in the dictionary
        boolean dictionaryHasWord = false;
        for(int i = 0; i < dictionary.length; i++){
            if(word.equals(dictionary[i])){
                dictionaryHasWord = true;
            }
        }
    }
}
```

Commenting: Java Docs!

“Javadoc is a tool which comes with JDK and it is used for generating Java code documentation in HTML format from Java source code”

- To use Java Docs, you have to use comments with a predefined format
- Comments start with `/**` and end with `*/`

```
/**  
 * The main entry point for this class  
 * @param args. A string array of arguments (not used)  
 */
```

Commenting: Java Docs!

"Javadoc is a tool which comes with JDK and it is used for generating Java code documentation in HTML format from Java source code"

- There are a set of pre-defined tags that you can use
 - @param
 - @return
 - @author (for class level Java Docs)
 - @version (for class level Java Docs)
 - etc ...
- Check them out here ... https://www.tutorialspoint.com/java/java_documentation.htm

Commenting: Java Docs!

“Javadoc is a tool which comes with JDK and it is used for generating Java code documentation in HTML format from Java source code”

- To generate your Java Doc files, in the terminal, use the command:

```
javadoc ClassName.java
```

or

```
javadoc *.java
```

Commenting: Java Docs!

- Let's try it ...

Refactoring

Refactoring

- Refactoring involves restructuring your code to make it:
 - Cleaner
 - Simpler
 - More readable
 - And to align with conventions (if it doesn't already)
- You should refactor often, not at the end when everything's written!
 - I.e. each time you finish a method, refactor it!
- Warning – there is a balance. You can refactor your code so much that it becomes unreadable or unnecessarily complex.

Refactoring

```
public class Main {

    private static String[] dictionary = {"time", "year", "people", "way", "day"};

    public static void main(String[] args){

        String testWord = "time";
        String word = testWord;

        boolean includesNumerical = false;
        for (int i = 0; i < word.length(); i++){
            if(Pattern.matches("[0-9]", String.valueOf(word.charAt(i)))){
                includesNumerical = true;
            }
        }

        if(includesNumerical){
            System.out.println("Sorry, there are numbers");
        }

        else {

            word = word.toLowerCase();
            word = word.replaceAll("[.!?\\-]", "");

            boolean dictionaryHasWord = false;
            for(int i = 0; i < dictionary.length; i++){
                if(dictionary[i].equals(word)){
                    dictionaryHasWord = true;
                }
            }
        }
    }
}
```

```
public class SpellChecker {

    ...

    public boolean checkIsWordReal(String word){
        if(this.includesNumerical(word)){
            return false;
        }

        word = this.convertToLower(word);
        word = this.stripPunctuation(word);
        return this.existsInDictionary(word);
    }
}
```

```
public class Main {

    public static void main(String[] args){

        SpellChecker spellChecker = new SpellChecker();
        System.out.println(spellChecker.checkIsWordReal("time"));

    }
}
```

Finally, a word of caution

A word of caution ... this isn't everything!

- For some more, check out 'code smells'

https://en.wikipedia.org/wiki/Code_smell#:~:text=In%20computer%20programming%2C%20a%20code,%2C%20developer%2C%20and%20development%20methodology.

Also, these are not hard and fast rules!

- Different languages use different conventions
 - Different papers may have different expectations
 - In industry, different companies will use different conventions
-
- But, this is a good start!

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

PRESS RECORD

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Software Construction Tools & Example

COMPX₃₄₁

Overview

- Software Construction Tools
- Example Product Backlog & Sprints

Software Construction Tools

Software Construction Tools

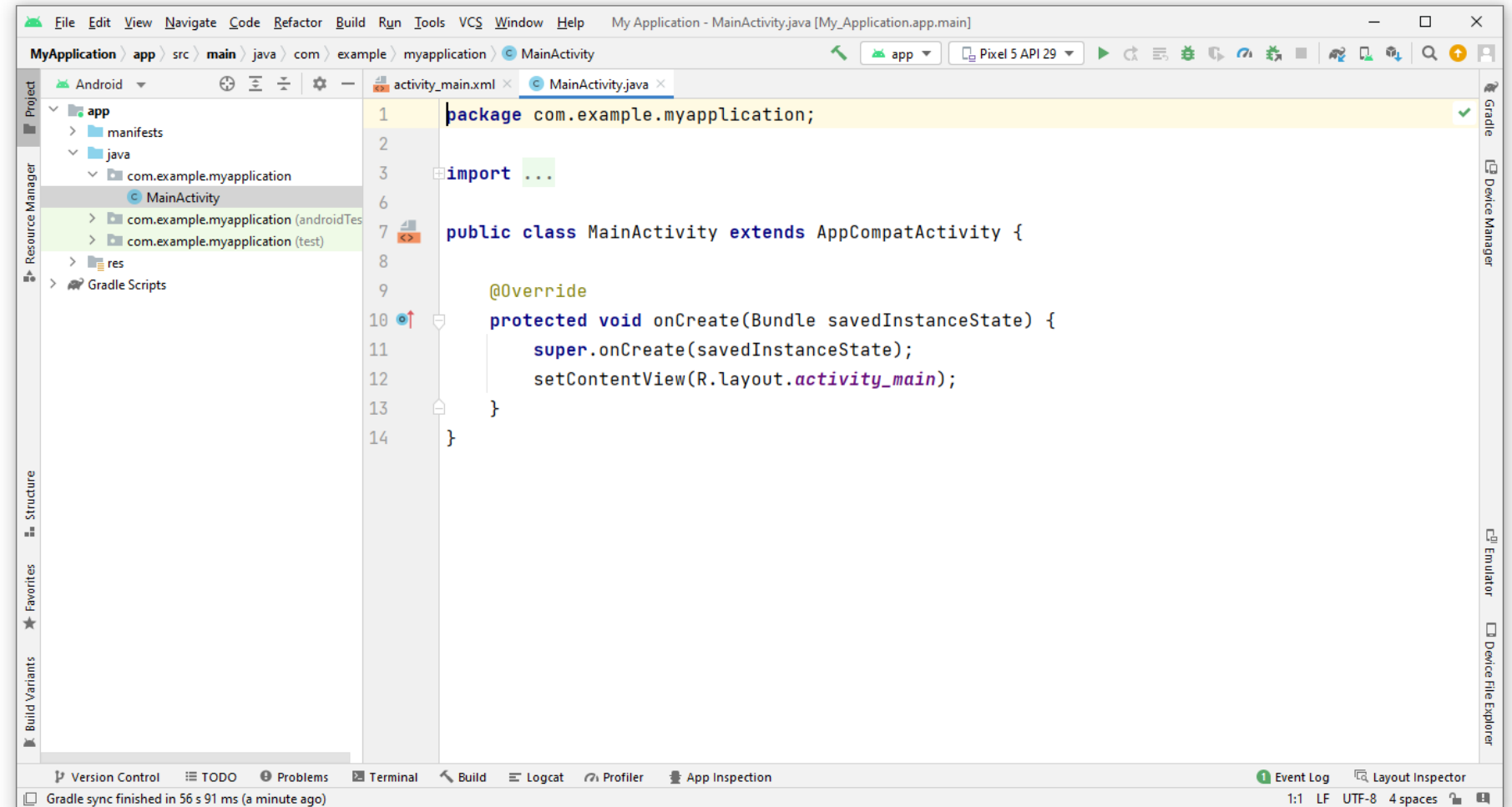
- Development Environments
- GUI Builders
- Unit Testing Tools
- Profiling, Performance Analysis, and Slicing Tools

Development Environments

- Development Environments (DE) or Integrated Development Environment (IDE)
 - Code editing
 - Compilation
 - Error detection
 - Integration with source code
 - Build/test/debugging tools
 - ...
- For example:
 - IntelliJ, Android Studio, etc.

Development Environments

- Code editing
- Compilation
- Error detection
- Integration with source code
- Build/test/debugging tools

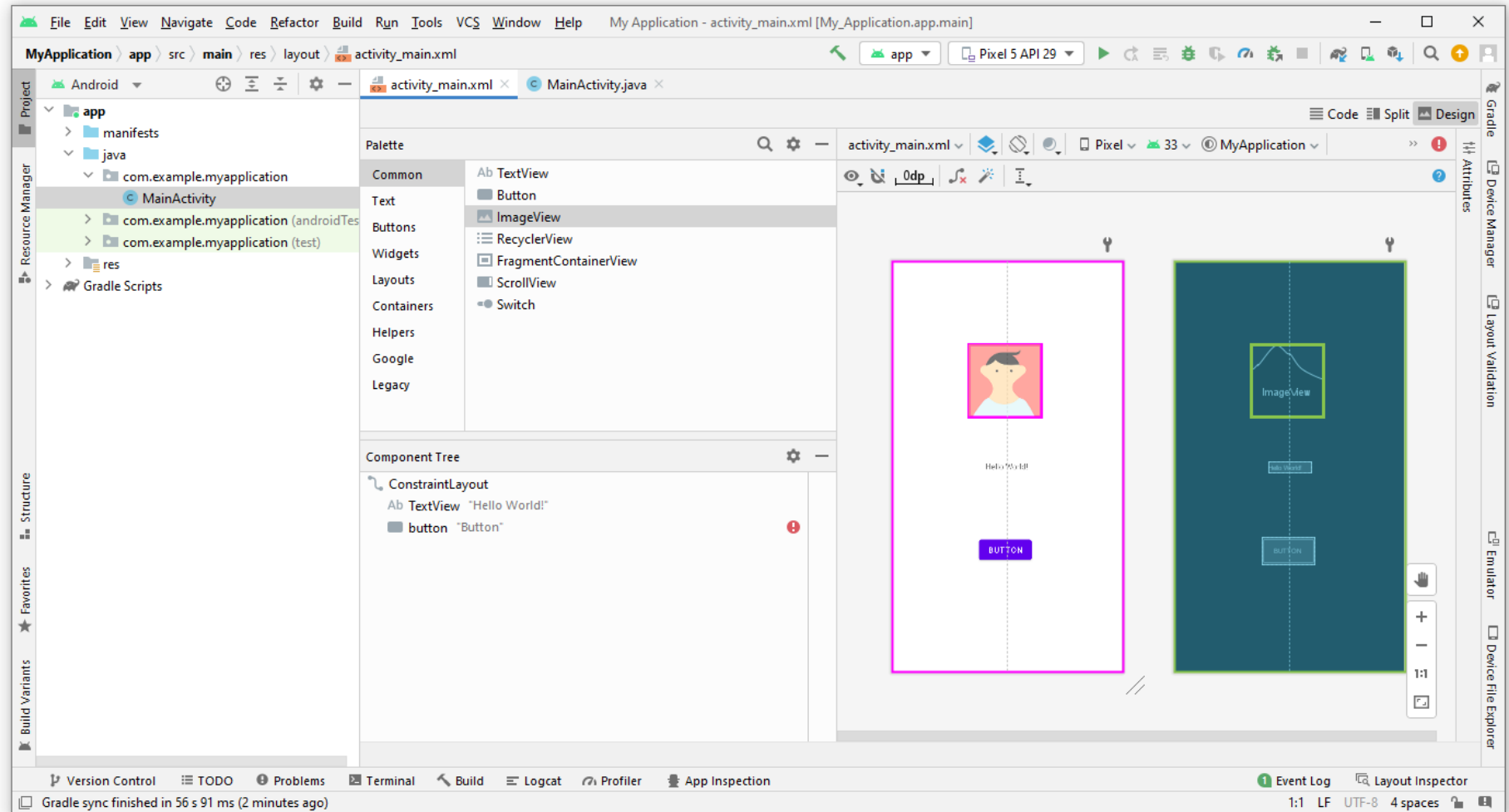


GUI Builders

- Allow you to maintain GUIs in a WYSIWYG mode
 - Visual editor
 - Drag-drop
 - Parameter setting
 - Automatic generation of source code (sometimes)
 - Code generation for event-driven development (sometimes)
 - Stand alone or integrated into an IDE
- For example:
 - Visual Studio, Android Studio, etc.

GUI Builders

- Visual editor
- Drag-drop
- Parameter setting
- Automatic generation of source code (sometimes)
- Code generation for event-driven development (sometimes)
- Stand alone or integrated into an IDE



Unit Testing Tools

- Allow you to extend and create automated testing environments
- For example:
 - JUnit, some IDEs

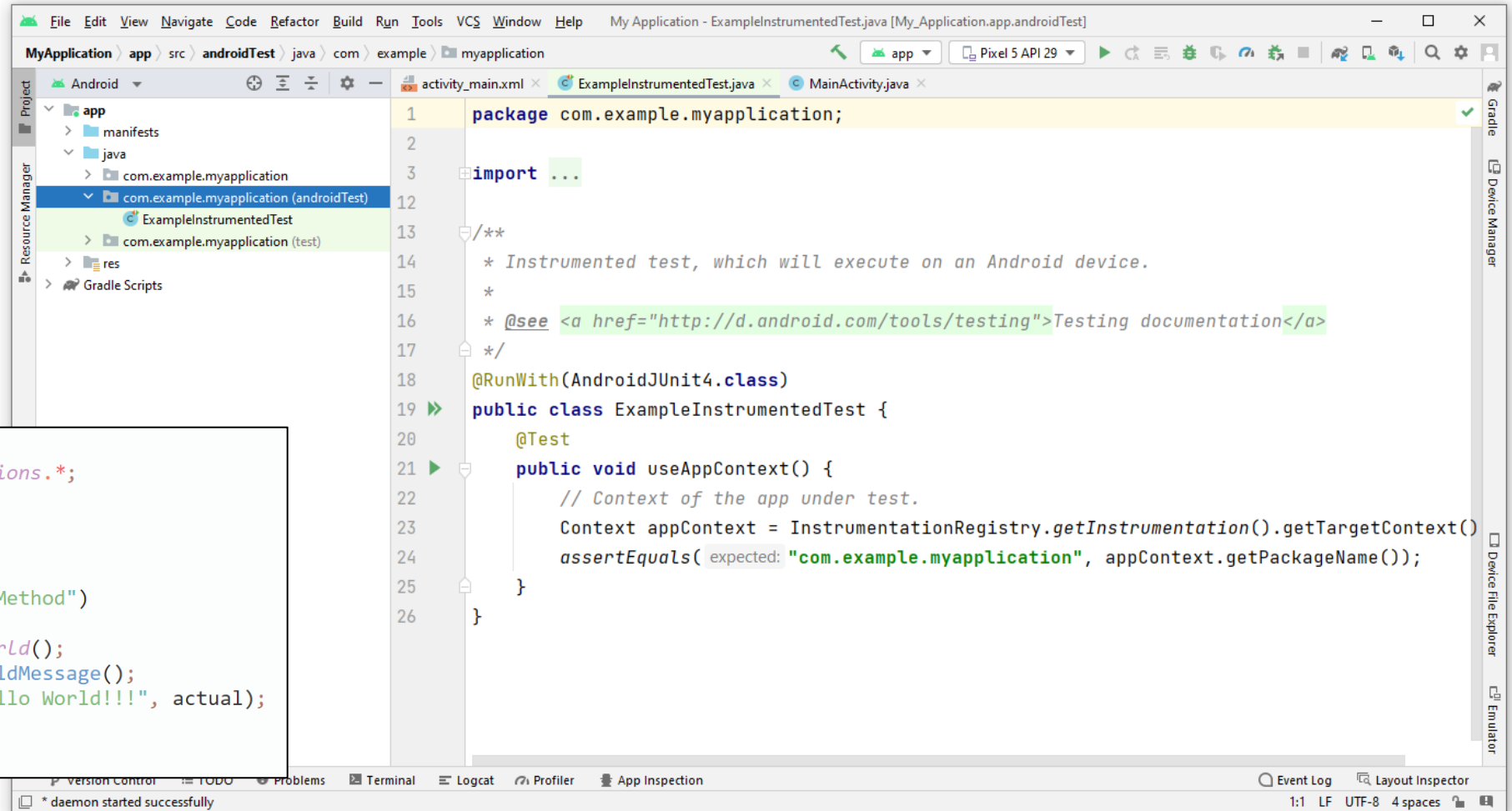
Unit Testing Tools

- Allow you to extend and create automated testing environments

```
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.Assertions.*;

class HelloWorldTest {

    @Test
    @DisplayName("Test Hello World Method")
    void helloWorldTest() {
        HelloWorld hw = new HelloWorld();
        String actual = hw.helloWorldMessage();
        Assertions.assertEquals("Hello World!!!", actual);
    }
}
```



Profiling, Performance Analysis, and Slicing Tools

- Allow you to monitor your code while it runs
 - Records how many times each statement is executed
 - Records how much time is spent on each statement or path
- For example:
 - https://en.wikipedia.org/wiki/List_of_performance_analysis_tools

Product Backlog and Sprint Example

Remember: Scrum

- Scrum is a lightweight, iterative and incremental framework for managing complex work.
 - Product Backlog (initial planning)
 - Sprints (planning, design, development, testing)

Remember: Product backlog

- Product backlog
 - A breakdown of the work that needs to be done
 - Product requirements
 - I.e. user stories, use cases
 - Define features, bug fixes, non-functional requirements
 - Controlled by the “product owner”

Remember: Sprint backlog

- Sprint backlog
 - The work to be addressed during the next sprint
 - Derived from items in the product backlog
 - Broken down into tasks
 - Team members “sign up” to complete a task
 - The full sprint backlog is not always completed in one sprint

SRS Excerpt

1 Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements of a software system called the New Zealand Pandemic Response Unit Tracker (NZPRU Tracker). This document will explain the capabilities of the software, its interface requirements, and both its functional and nonfunctional requirements.

1.2 Project Scope

The NZPRU Tracker is a software-intensive project to assist in containing epidemic outbreaks. During high-risk periods, people's visits to pharmacies, cafes, restaurants, supermarkets and other venues needs to be tracked. By doing so, if a person is suspected to be infected, NZPRU will be able to immediately identify all others who might have been infected by visiting the same venue at the same time.

NZPRU plans on deploying off-the-shelf tablets leveraging the venues' WiFi connectivity for collecting the remote data. Visitors to venues will be using the tablets to input their data when they enter as well as inputting the time they left on their way out of the venue

2 Overall Description

2.1 User Classes and Characteristics

NZPRU has three types of users.

- Community users: Community users do not need to login. Community users can use the mobile application to specify when they entered and exited a venue.
- Venue users: Venue users can login to the mobile application. They will login at the start of each day and logout at the end of each day. This will give the community users access to the application.
- NZPRU System administrators: Administrators can login to the web-based front end. They can view statistics, manage users.

SDS Excerpt

This will be first page shown when the application opens. In this page it will allow the installer to login with their username and password.

After he/she has login, it will allow the installer to enter the venue's details such as name and address. When this detail has been submitted it will jump to the welcome page

User ID:

Password:

Login

Information Of Venue

Name:

Address:

City:

Submit

SDS Excerpt

[Logout](#)

Welcome To
[venue Name]

Enter

Exit

Change Details

 English

 Maori

 Chinese

 Spanish

 French

 Korean

User ID: 123456

Password:

Logout

The installer is able to logout the venue through the logout button at top right corner. The logout page will provide the installer's id number and only require the installer to enter their password to logout. The logout is use for venues that will no longer needs this tablet/software.

SDS Excerpt

Are you visiting as

Individual ☒

Group ☐

Submit

For individuals, they will enter their personal details in this page such as name, email, phone number and address.

For the group, it will require one person from the group to enter their personal details and select how many people are in the group that came together on the day. For each group member they will only need to enter their first and last name. This will benefit the user without needing to line up outside for a long time waiting to enter their details in and also to help children or elderly who does not have access to phone number or email address.

SDS Excerpt

Individual

First Name:

Last Name:

Phone:

Email:

Address:

City:

Group

First Name:

Last Name:

Phone:

Email:

Address:

City:

Group Size:

1. First Name:

Last Name:

2. First Name:

Last Name:

The user will not need to enter the time and date that they have entered the venue as the system will automatically get these details and store it in database with the users' details, by doing this can save time for user when entering their details.

After the user has submitted their details it will jump to a new page with a given id number, this number will be used for reporting their exit of the venue when they leave. This number will also be send to the user's phone number and email address that they have provided in the previous page, as it can prevent the user of forgetting this number.

SDS Excerpt

ID Number

0001

*Please keep this number for reporting exit

Close

This page will automatically be closed in 20 seconds in case if the user has forgotten to close this page. When this page is closed it will go back to the welcome page.

SDS Excerpt

ID Number ☒	Please select the name that you want to exit ☐ Select All ☒ Person 1 ☒ Person 2 ☐ Person 3
Phone Number ☐	
Submit	Submit

The user will not need to enter the time and date that they leave the venue as the system will automatically get these details and store it in database with the users' details.

After they have reported their exit, it will jump to a thankyou page and this page will automatically be closed in 20 seconds in case if the user has forgotten to close this page. When this page is closed it will go back to the welcome page.

Remember: Product backlog

- Product backlog
 - A breakdown of the work that needs to be done
 - Product requirements
 - I.e. user stories, use cases
 - Define features, bug fixes, non-functional requirements
 - Controlled by the “product owner”

Product backlog example

- What are our features?
 - MoSCoW
1. Venue user tablet access
 2. Community user tablet interaction
 3. Back-end server data update
 4. ...

Product backlog example

1. Venue user tablet access

Sally Sallyson owns a café in Hamilton. Her café is only small, but is in the CBD and has great Wi-Fi coverage. She is keen to get involved in the NZPRU Tracking initiative, so she has volunteered to be one of the first venues to use the Mobile application. As a Venue User, Sally would need to be able to login and logout of the mobile application.

Frank Frankson owns a restaurant in rural Waikato. His restaurant is only small, but it is very popular with the locals. Unfortunately, the rural location means that Frank's internet connection is unstable. He is keen to be involved in the NZPRU Tracking initiative but he's not sure if his location is appropriate. As a Venue User, Frank would need to be able to login and logout of the mobile application.

Betty Bettyson owns a plant shop in Hamilton. Her plant shop is quite big, and it has a front and a back entrance. She is keen to get involved in the NZPRU Tracking initiative, so she has volunteered to be one of the first venues to use the Mobile application. She will need a tablet at each entrance to her shop. As a Venue User, Betty would need to be able to login and logout of the mobile application.

Product backlog example

2. Community user tablet interaction

Sarah Sarahson goes out to her local café at the same time every morning to get a cup of coffee. She is happy to enter her information into the NZPRU tracker, but she's always in a rush so the faster and easier it can be, the better. As a community user, Sarah would need to be able to register her entry and exit from the venue.

Harry Harryson is going out to a restaurant with his daddy-and-me group. There are four adults and seven children who join them every week. They are happy to enter their information into the NZPRU tracker, but do not want to enter information for all of the children individually. As a community user, Harry and his group would need to be able to register their entry and exit from the venue.

... one that is not happy to share personal information?

Remember: Sprint backlog

- Sprint backlog
 - The work to be addressed during the next sprint
 - Derived from items in the product backlog
 - Broken down into tasks
 - Team members “sign up” to complete a task
 - The full sprint backlog is not always completed in one sprint

Sprint backlog example

Venue user tablet access

Sally Sallyson owns a café in Hamilton. Her café is only small, but is in the CBD and has great Wi-Fi coverage. She is keen to get involved in the NZPRU Tracking initiative, so she has volunteered to be one of the first venues to use the Mobile application. As a Venue User, Sally would need to be able to login and logout of the mobile application.

1. (Development) Venue user login on tablet
2. (Development) Venue user login validation on back-end
3. (Development) Venue user logout on tablet

Sprint backlog example

Venue user tablet access

Frank Frankson owns a restaurant in rural Waikato. His restaurant is only small, but it is very popular with the locals. Unfortunately, the rural location means that Frank's internet connection is unstable. He is keen to be involved in the NZPRU Tracking initiative but he's not sure if his location is appropriate. As a Venue User, Frank would need to be able to login and logout of the mobile application.

1. (Development) Venue user login on tablet
2. (Development) Venue user login validation on back-end
3. (Development) Venue user logout on tablet
4. (Requirement/Design) Decision on supporting unstable internet (caching etc.)

Sprint backlog example

Venue user tablet access

Betty Bettyson owns a plant shop in Hamilton. Her plant shop is quite big, and it has a front and a back entrance. She is keen to get involved in the NZPRU Tracking initiative, so she has volunteered to be one of the first venues to use the Mobile application. She will need a tablet at each entrance to her shop. As a Venue User, Betty would need to be able to login and logout of the mobile application.

1. (Development) Venue user login on tablet
2. (Development) Venue user login validation on back-end
3. (Development) Venue user logout on tablet
4. (Development) Allow multiple logins at once (back-end)

Sprint backlog example

Community user tablet interaction

Sarah Sarahson goes out to her local café at the same time every morning to get a cup of coffee. She is happy to enter her information into the NZPRU tracker, but she's always in a rush so the faster and easier it can be, the better. As a community user, Sarah would need to be able to register her entry and exit from the venue.

1. (Development) Register individual entry into venue
2. (Development) Register individual exit out of venue
3. (Requirement/Design) Simple UI, Fast processing

Sprint backlog example

Community user tablet interaction

Harry Harryson is going out to a restaurant with his daddy-and-me group. There are four adults and seven children who join them every week. They are happy to enter their information into the NZPRU tracker, but do not want to enter information for all of the children individually. As a community user, Harry and his group would need to be able to register their entry and exit from the venue.

1. (Development) Register group entry into venue
2. (Development) Register group exit out of venue
3. (Development) Register children entry and exit

Putting it all together

- At the start:
 - Product Backlog for all (or MVP) features/requirements
- Next:
 - Sprint #1
 - Select items from product backlog
 - Break them down into tasks
 - Work through them until your sprint is finished
 - Sprint #2
 - Carry over any unfinished tasks from the last sprint
 - Select more items from the product backlog
 - Break them down into tasks
 - Work through them until your sprint is finished
 - Sprint #3
 - ...

At the start: product backlog

1. Venue user tablet access

Sally Sallyson owns a café in Hamilton. Her café is only small, but is in the CBD and has great Wi-Fi coverage. She is keen to get involved in the NZPRU Tracking initiative, so she has volunteered to be one of the first venues to use the Mobile application. As a Venue User, Sally would need to be able to login and logout of the mobile application.

Frank Frankson owns a restaurant in rural Waikato. His restaurant is only small, but it is very popular with the locals. Unfortunately, the rural location means that Frank's internet connection is unstable. He is keen to be involved in the NZPRU Tracking initiative but he's not sure if his location is appropriate. As a Venue User, Frank would need to be able to login and logout of the mobile application.

Betty Bettyson owns a plant shop in Hamilton. Her plant shop is quite big, and it has a front and a back entrance. She is keen to get involved in the NZPRU Tracking initiative, so she has volunteered to be one of the first venues to use the Mobile application. She will need a tablet at each entrance to her shop. As a Venue User, Betty would need to be able to login and logout of the mobile application.

2. Community user tablet interaction

Sarah Sarahson goes out to her local café at the same time every morning to get a cup of coffee. She is happy to enter her information into the NZPRU tracker, but she's always in a rush so the faster and easier it can be, the better. As a community user, Sarah would need to be able to register her entry and exit from the venue.

Harry Harryson is going out to a restaurant with his daddy-and-me group. There are four adults and seven children who join them every week. They are happy to enter their information into the NZPRU tracker, but do not want to enter information for all of the children individually. As a community user, Harry and his group would need to be able to register their entry and exit from the venue.

3. Next Feature/Requirement

...

Next: Sprint 1

Stories selected from Product Backlog:

Sally Sallyson owns a café in Hamilton. Her café is only small, but is in the CBD and has great Wi-Fi coverage. She is keen to get involved in the NZPRU Tracking initiative, so she has volunteered to be one of the first venues to use the Mobile application. As a Venue User, Sally would need to be able to login and logout of the mobile application.

Frank Frankson owns a restaurant in rural Waikato. His restaurant is only small, but it is very popular with the locals. Unfortunately, the rural location means that Frank's internet connection is unstable. He is keen to be involved in the NZPRU Tracking initiative but he's not sure if his location is appropriate. As a Venue User, Frank would need to be able to login and logout of the mobile application.

Betty Bettyson owns a plant shop in Hamilton. Her plant shop is quite big, and it has a front and a back entrance. She is keen to get involved in the NZPRU Tracking initiative, so she has volunteered to be one of the first venues to use the Mobile application. She will need a tablet at each entrance to her shop. As a Venue User, Betty would need to be able to login and logout of the mobile application.

Sprint Backlog:

1. (Development) Venue user login on tablet
2. (Development) Venue user login validation on back-end
3. (Development) Venue user logout on tablet
4. (Requirement/Design) Decision on supporting unstable internet (caching etc.)
5. (Development) Allow multiple logins at once (back-end)

Next: Sprint 2

Stories selected from Product Backlog:

Sarah Sarahson goes out to her local café at the same time every morning to get a cup of coffee. She is happy to enter her information into the NZPRU tracker, but she's always in a rush so the faster and easier it can be, the better. As a community user, Sarah would need to be able to register her entry and exit from the venue.

Harry Harryson is going out to a restaurant with his daddy-and-me group. There are four adults and seven children who join them every week. They are happy to enter their information into the NZPRU tracker, but do not want to enter information for all of the children individually. As a community user, Harry and his group would need to be able to register their entry and exit from the venue.

Sprint Backlog:

1. (Carried over) (Development) Allow multiple logins at once (back-end)
2. (Development) Register individual entry into venue
3. (Development) Register individual exit out of venue
4. (Requirement/Design) Simple UI, Fast processing
5. (Development) Register group entry into venue
6. (Development) Register group exit out of venue
7. (Development) Register children entry and exit

Next: Sprint 3

Stories selected from Product Backlog:

...

Sprint Backlog:

1. (Carried over) (Development) Register group entry into venue
2. (Carried over)(Development) Register group exit out of venue
3. (Carried over) (Development) Register children entry and exit
4. ...

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

PRESS RECORD

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

FYI – Soft Schedule

	Lecture 1	Lecture 2	Lecture 3	Assessments
Week 7	Testing w Steve	ANZAC Day	Construction Git and GitLab	
Week 8	Construction Life Cycle Models	Construction Software Quality	Guest Speaker Sarah Mann	CW2 Due CW3 Released
Week 9	Construction Code Quality	Construction Tools and Example	Construction GraphStream Example	
Week 10	Maintenance An Introduction	Maintenance Analysis and Testing (PRE-RECORD)	Maintenance Tools and Example	CW3 Due CW4 Released
Week 11	DevOps An Introduction	DevOps CICD Example	Guest Speaker Liam Clow	
Week 12	DevOps Measuring and Embracing Risk	TBD	TEST 2	CW4 Due

GraphStream

COMPX₃₄₁

GraphStream Demonstration

- Compiling and running GraphStream

```
javac -cp "gs-core-1.3.jar;gs-ui-1.3.jar" *.java  
java -cp ".;gs-core-1.3.jar;gs-ui-1.3.jar" Main
```

GraphStream Demonstration

- Create a super basic Graph
 - <https://graphstream-project.org/doc/Tutorials/Getting-Started/>
- Create a Graph using the Device IDs
 - <https://graphstream-project.org/doc/Tutorials/Getting-Started/>
- Create a Graph using the Device IDs & store Device objects within Nodes
 - <https://graphstream-project.org/doc/Tutorials/Storing-retrieving-and-displaying-data-in-graphs/>
- Experiment with a stylesheet for the Graph
 - <https://graphstream-project.org/doc/Tutorials/Graph-Visualisation/>
 - <https://graphstream-project.org/doc/Advanced-Concepts/GraphStream-CSS-Reference/>
- Experiment with a stylesheet for Nodes
 - <https://graphstream-project.org/doc/Tutorials/Graph-Visualisation/>
 - <https://graphstream-project.org/doc/Advanced-Concepts/GraphStream-CSS-Reference/>
- Experiment with a stylesheet for Edges
 - <https://graphstream-project.org/doc/Tutorials/Graph-Visualisation/>
 - <https://graphstream-project.org/doc/Advanced-Concepts/GraphStream-CSS-Reference/>

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

PRESS RECORD

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Software Maintenance – an introduction

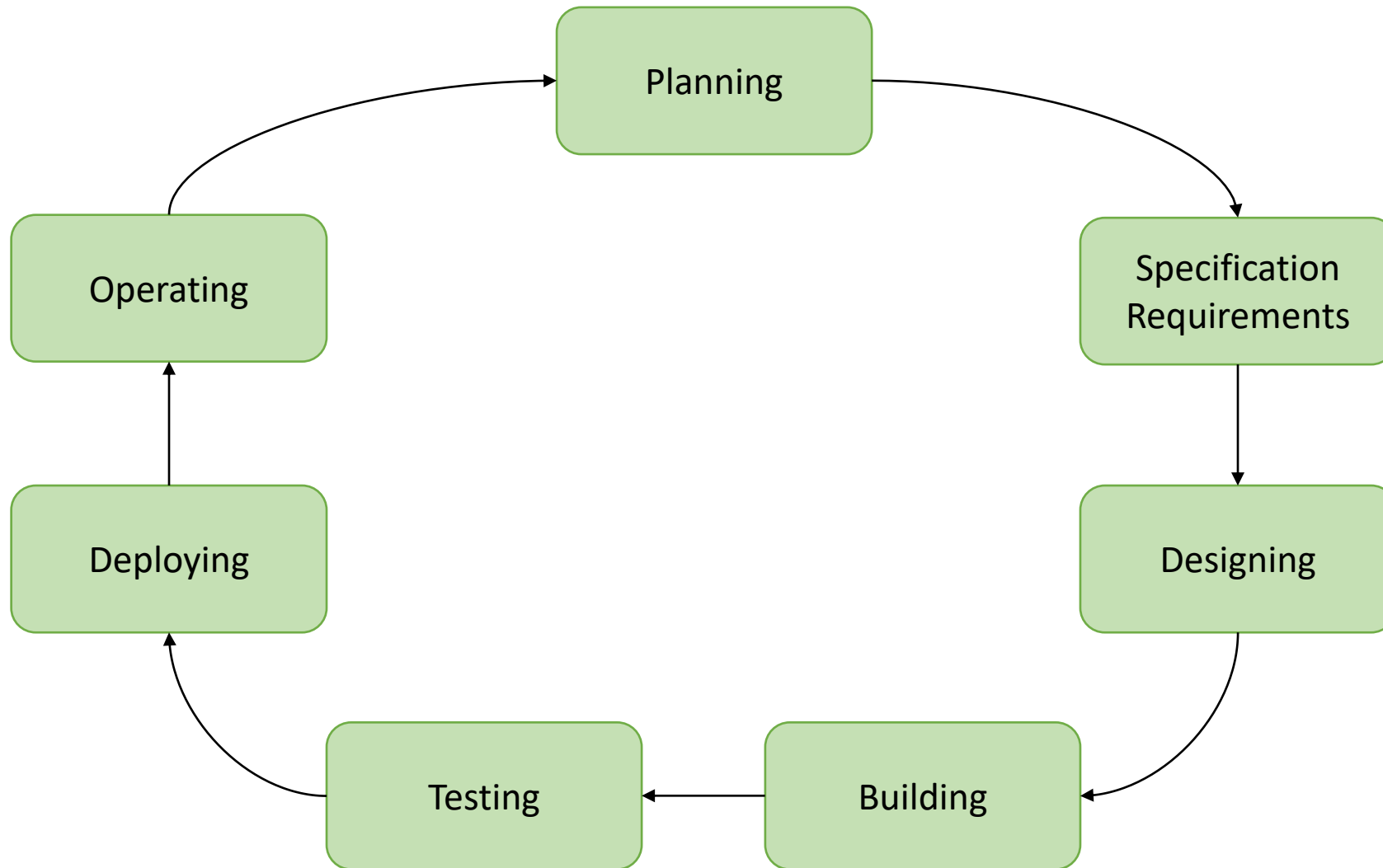
COMPX₃₄₁

Overview

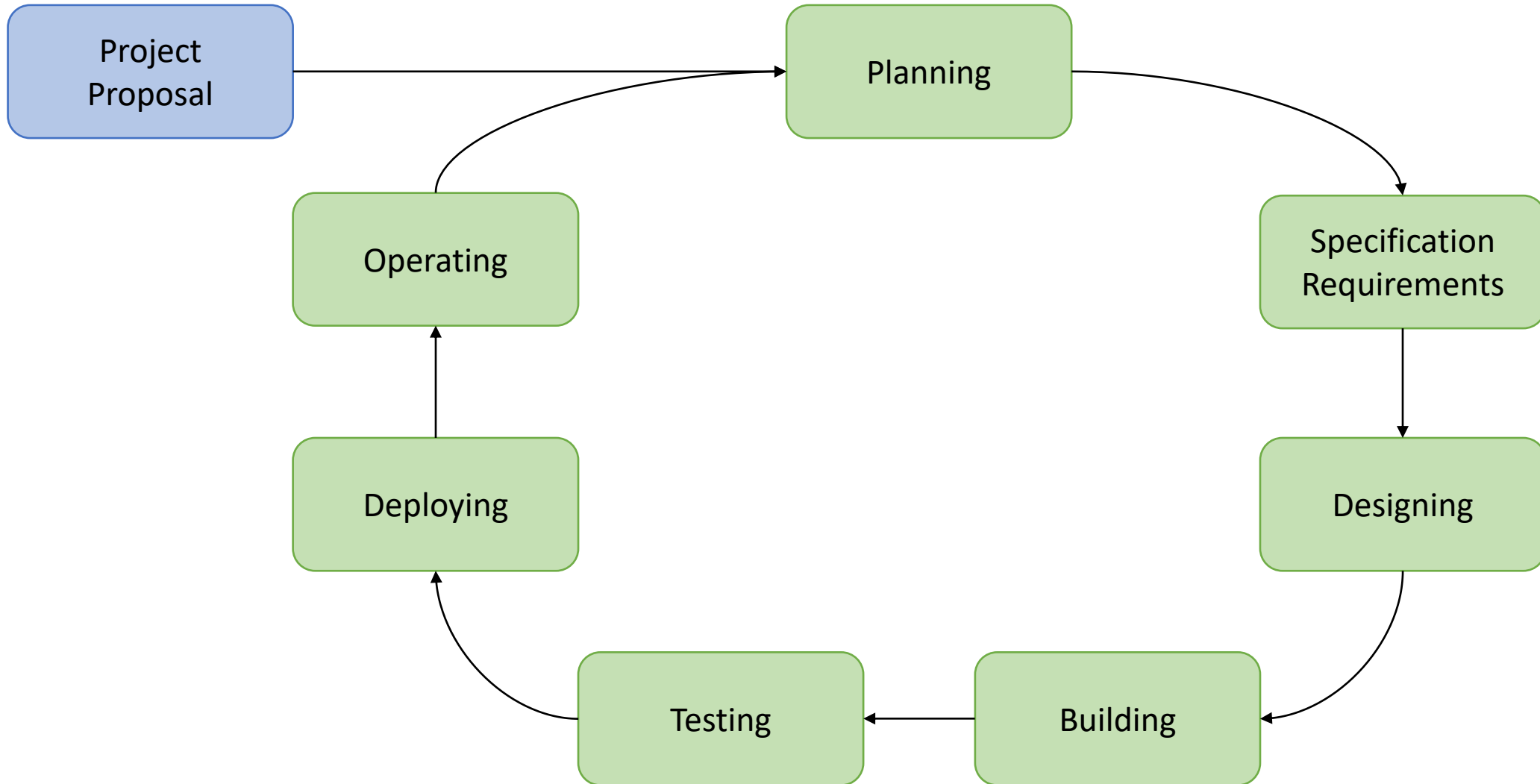
- Software lifecycle
- Software maintenance
- Self-adaptive Systems

Software Life Cycle

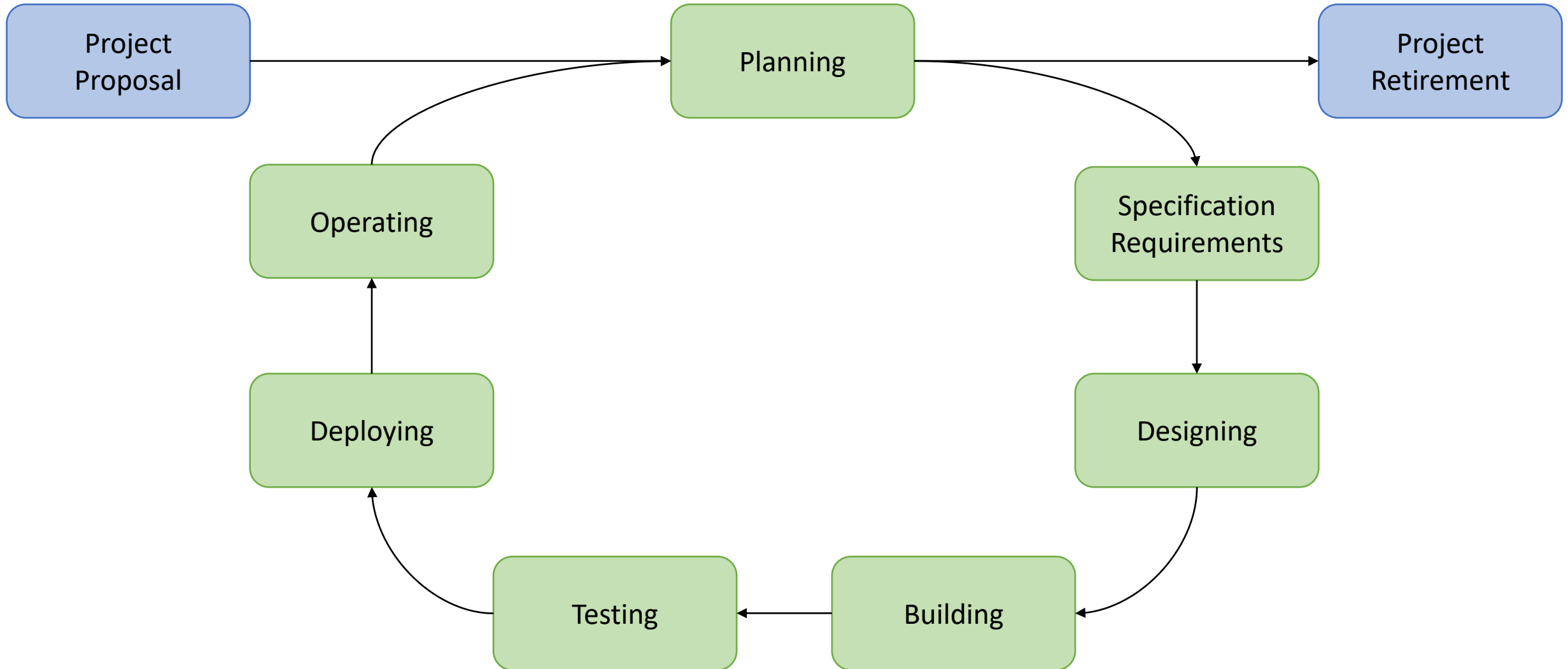
Software Life Cycle



Software Life Cycle



Software Life Cycle

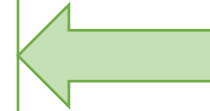


Software Life Cycle

- > Project proposal
- > 1st iteration (planning, requirements, ..., deployment)
- > Operation
- > 2nd iteration (planning, requirements, ..., deployment)
- > Operation
- ...
- > Nth iteration (planning, requirements, ..., deployment)
- > Operation
- > Retirement

Software Life Cycle

- > Project proposal
- > 1st iteration (planning, requirements, ..., deployment)
- > Operation
- > 2nd iteration (planning, requirements, ..., deployment)
- > Operation
- ...
- > Nth iteration (planning, requirements, ..., deployment)
- > Operation
- > Retirement



Software Maintenance
describes these parts
of the Software Life Cycle

Software Maintenance

Software Maintenance

“... all the activities required to provide cost-effective support to software”

-- SWEBOK v3

“The objective of Software Maintenance is to modify existing software while preserving its integrity”

-- SWEBOK v3

- Essentially, modifications after delivery
- Conducted by maintainer

Need for Software Maintenance

1. Correct faults
2. Improve the design
3. Implement enhancements
4. Interface with other software
5. Adapt programs so that different hardware, software, system features, and telecommunication facilities can be used
6. Migrate legacy software
7. Retire software
8. Maintaining control over the software's day-to-day functions
9. Maintaining control over software modification
10. Perfecting existing functions
11. Identifying security threats and fixing security vulnerabilities
12. Preventing software performance from degrading to unacceptable levels

Reasons for Maintenance

- Correction
 - Same requirements
 - E.g. Correcting an existing feature based on existing requirements
- Enhancement
 - Changes requirements and/or changed environment
 - E.g. Adding a new feature before of changes to the requirments

Times for Maintenance

- Proactive
 - To prevent failure
 - E.g. making the change prior to failure occurring
 - Failure can refer to feature failure, system failure, or failure to meet requirements
- Reactive
 - Because of one or more reported failures
 - E.g. making the change after a failure has occurred (and been reported)
 - Failure can refer to feature failure, system failure, or failure to meet requirements

Software Maintenance Categories

- Preventative
- Perfective
- Corrective
- Adaptive

Software Maintenance Categories

	Correction Same requirements	Enhancement Changes requirements and/or changed environment
Proactive To prevent failure	Preventive Bugs are there but not surfacing ... yet!	Perfective New features or documentation; better performance; etc.
Reactive Because of one or more reported failures	Corrective a.k.a. Repairs Special case: emergency	Adaptive Changed/changing environment (e.g., updated OS, middleware, etc.)

Software Maintenance Categories

	Correction	Enhancement
	Same requirements	Changes requirements and/or changed environment
Proactive	Preventive  Bugs are there but not surfacing ... yet!	Perfective  New features or documentation; better performance; etc.
Reactive	Corrective  a.k.a. Repairs Special case: emergency	Adaptive  ~   Changed/changing environment (e.g., updated OS, middleware, etc.)

Examples: what kind of maintenance is required?

Issue description	Preventative Proactive- correction	Perfective Proactive- enhancement	Corrective Reactive- correction	Adaptive Reactive- enhancement
One of our testers realised the deployed product could crash if a user logs in from two devices at the same time				
It's been recommended we add a bar chart plot to this report				
The web app is getting slower and slower now that we've increased our client base so much!				
The client called as at 2am – one hour ago. “I have no clue why all these train tickets were sold at half the price.”				

Examples: what kind of maintenance is required?

Issue description	Preventative Proactive- correction	Perfective Proactive- enhancement	Corrective Reactive- correction	Adaptive Reactive- enhancement
One of our testers realised the deployed product could crash if a user logs in from two devices at the same time	✓			
It's been recommended we add a bar chart plot to this report				
The web app is getting slower and slower now that we've increased our client base so much!				
The client called as at 2am – one hour ago. “I have no clue why all these train tickets were sold at half the price.”				

Examples: what kind of maintenance is required?

Issue description	Preventative Proactive- correction	Perfective Proactive- enhancement	Corrective Reactive- correction	Adaptive Reactive- enhancement
One of our testers realised the deployed product could crash if a user logs in from two devices at the same time	✓			
It's been recommended we add a bar chart plot to this report		✓		
The web app is getting slower and slower now that we've increased our client base so much!				
The client called as at 2am – one hour ago. “I have no clue why all these train tickets were sold at half the price.”				

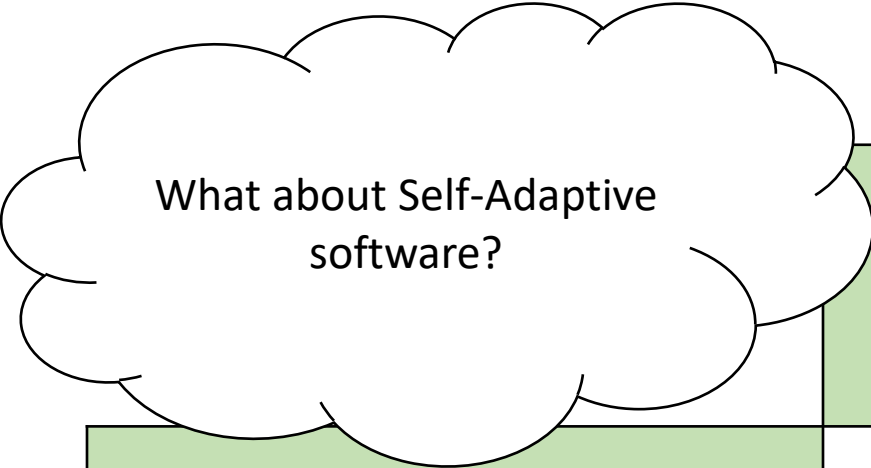
Examples: what kind of maintenance is required?

Issue description	Preventative Proactive- correction	Perfective Proactive- enhancement	Corrective Reactive- correction	Adaptive Reactive- enhancement
One of our testers realised the deployed product could crash if a user logs in from two devices at the same time	✓			
It's been recommended we add a bar chart plot to this report		✓		
The web app is getting slower and slower now that we've increased our client base so much!				✓
The client called as at 2am – one hour ago. “I have no clue why all these train tickets were sold at half the price.”				

Examples: what kind of maintenance is required?

Issue description	Preventative Proactive- correction	Perfective Proactive- enhancement	Corrective Reactive- correction	Adaptive Reactive- enhancement
One of our testers realised the deployed product could crash if a user logs in from two devices at the same time	✓			
It's been recommended we add a bar chart plot to this report		✓		
The web app is getting slower and slower now that we've increased our client base so much!				✓
The client called as at 2am – one hour ago. “I have no clue why all these train tickets were sold at half the price.”			✓	

Software Maintenance Categories



	Correction	Enhancement
	Same requirements	Changes requirements and/or changed environment
Proactive	Preventive	Perfective
To prevent failure	Bugs are there but not surfacing ... yet!	New features or documentation; better performance; etc.
Reactive	Corrective	Adaptive
Because of one or more reported failures	a.k.a. Repairs Special case: emergency	Changed/changing environment (e.g., updated OS, middleware, etc.)

What is self-adaptive software?

What is self-adaptive software?

- Software that tries to maintain itself!

What is self-adaptive software?

- Software that tries to maintain itself!
 - i.e. self-configuration: changes configuration constants
 - i.e. self-optimization: changes optimization parameters
 - i.e. self-protection module: filters external attacks
 - i.e. self-healing: ready-made mechanisms to ensure the system continues operating as intended
- E.g. auto-scaling automatically looks at and adapts the system such that it has the ideal number of instances
- E.g. cloud often has the capacity to bring up an application that has crashed (self-healing)

What is self-adaptive software?

- Note: the maintenance team is still there!
- Self-adaptive systems just try to minimise their involvement

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

PRESS RECORD

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Software Maintenance – Analysis and Testing

COMPX₃₄₁

Overview

- Key issues in software maintenance
- Maintenance process
- Techniques for Maintenance

Key Issues in Software Maintenance

Key Issues in Software Maintenance

- Technical Issues
- Management Issues
- Maintenance Cost Estimation
- Software Maintenance Measurement

Key Issues in Software Maintenance

Technical Issues:

- Limited understanding

“Research indicates that about half of the total maintenance effort is devoted to understanding the software to be modified.”
- Testing

“The cost of repeating full testing on a major piece of software is significant in terms of time and money.”
- Impact Analysis

“Impact analysis describes how to conduct, cost effectively, a complete analysis of the impact of a change in existing software.”

...

Key Issues in Software Maintenance

Technical Issues (continued):

- Impact Analysis

Note, probably not for emergencies

1. analyse the modification request (MR) / problem request (PR);
2. replicate or verify the problem;
3. develop options for implementing the modification;
4. document the MR/PR, the results, and the execution options;
5. obtain approval for the selected modification option.

Key Issues in Software Maintenance

Technical Issues (continued):

- Limited understanding

“Research indicates that about half of the total maintenance effort is devoted to understanding the software to be modified.”
- Testing

“The cost of repeating full testing on a major piece of software is significant in terms of time and money.”
- Impact Analysis

“Impact analysis describes how to conduct, cost effectively, a complete analysis of the impact of a change in existing software.”
- Maintainability

“The capability of the software product to be modified.”

Key Issues in Software Maintenance

Management issues

- Alignment with organizational objectives
 - “the view at the senior management level is often that of a major activity consuming significant resources with no clear quantifiable benefit for the organization.”
- Staffing
 - “maintenance personnel are frequently viewed as “second-class citizens,” and morale therefore suffers”
- Process
 - “Maintenance also requires several activities that are not found in software development ... These activities present challenges to management.”
- Organizational aspects of maintenance
 - “how to identify which organization and/or function will be responsible for the maintenance of software.”
- Outsourcing

Key Issues in Software Maintenance

Maintenance cost estimation

- Cost estimation

“how impact analysis identifies all systems and software products affected by a software change request and develops an estimate of the resources needed to accomplish that change.”

How? By using parametric models and expert experience.

- Parametric Models

“historical data from past maintenance are needed in order to use and calibrate the mathematical models.”

- Experience

“Expert judgment is often used to estimate maintenance effort.”

Key Issues in Software Maintenance

Software Maintenance Measurement

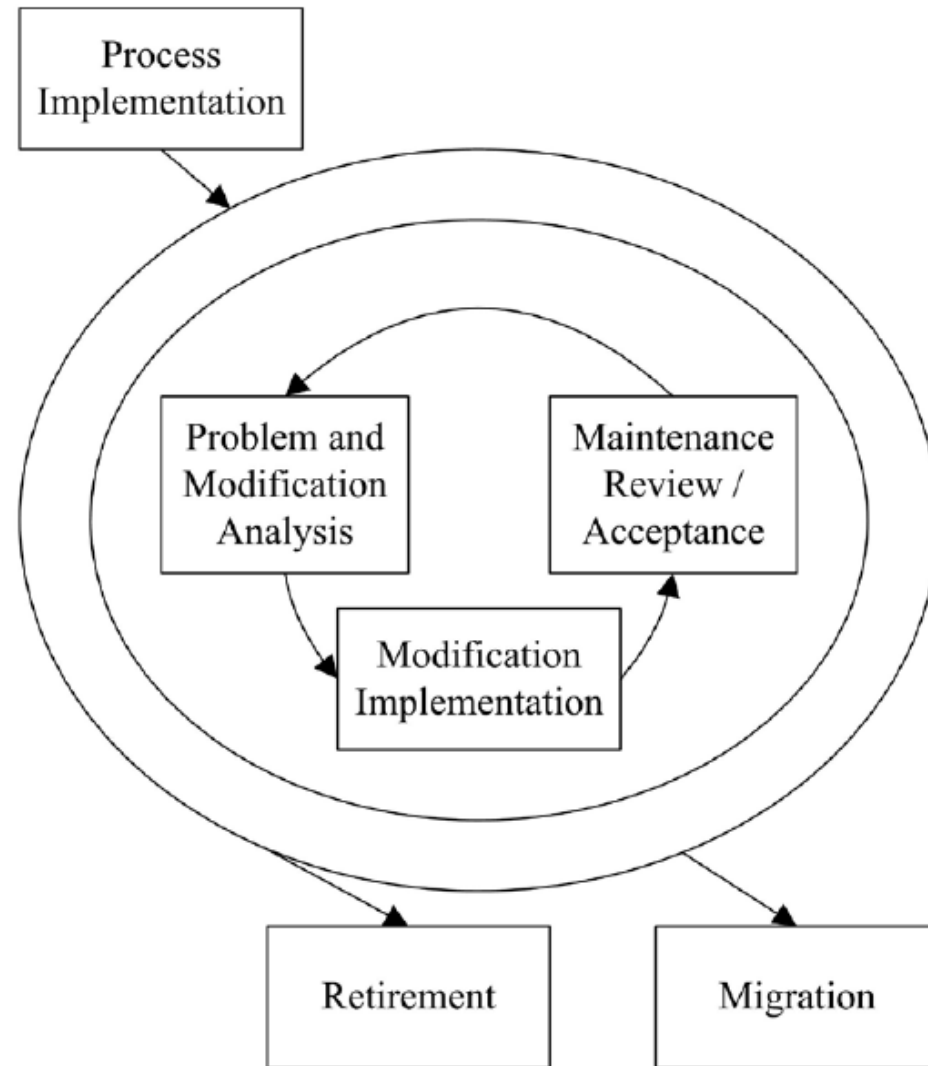
- Analyzability
How difficult (effort and resources) was it for the maintainer to diagnose a problem or identify a modification?
- Changeability
How difficult (effort and resources) was it for a maintainer to implement a modification?
- Stability
How much unexpected behavior did the system exhibit? Before and after modification, including testing.
- Testability
How difficult (effort and resources) was it for the maintainer (and user) to test the modified software?
- Other
Software size, complexity, understandability, maintainability

Maintenance Process

Maintenance process

- The Maintenance lifecycle includes a series of activities, some of which we may not have seen before
 - Process implementation
 - Problem and modification analysis
 - Modification implementation
 - Maintenance review/acceptance
 - Migration
 - Software retirement

Maintenance process



Maintenance process

- Within this lifecycle, some activities will be familiar
 - Analysis
 - Design
 - Coding
 - Testing
 - Documentation
- But some activities are unique to maintenance ...

Maintenance process

Unique Activities

- Program understanding
- Transition
 - Transferring software from developer to maintainer
- Modification request acceptance/rejection
 - Size/effort/complexity of modifications
 - Maintainer versus developer
- Maintenance help desk
 - Assessment, prioritization, and costing of modification requests
- Impact analysis
- Maintenance Service-Level Agreements (SLAs)
 - Contractual obligations between the client and the software provider

Techniques for Maintenance

Techniques for Maintenance

- Program comprehension
- Re-engineering
- Reverse Engineering
- Migration
- Retirement

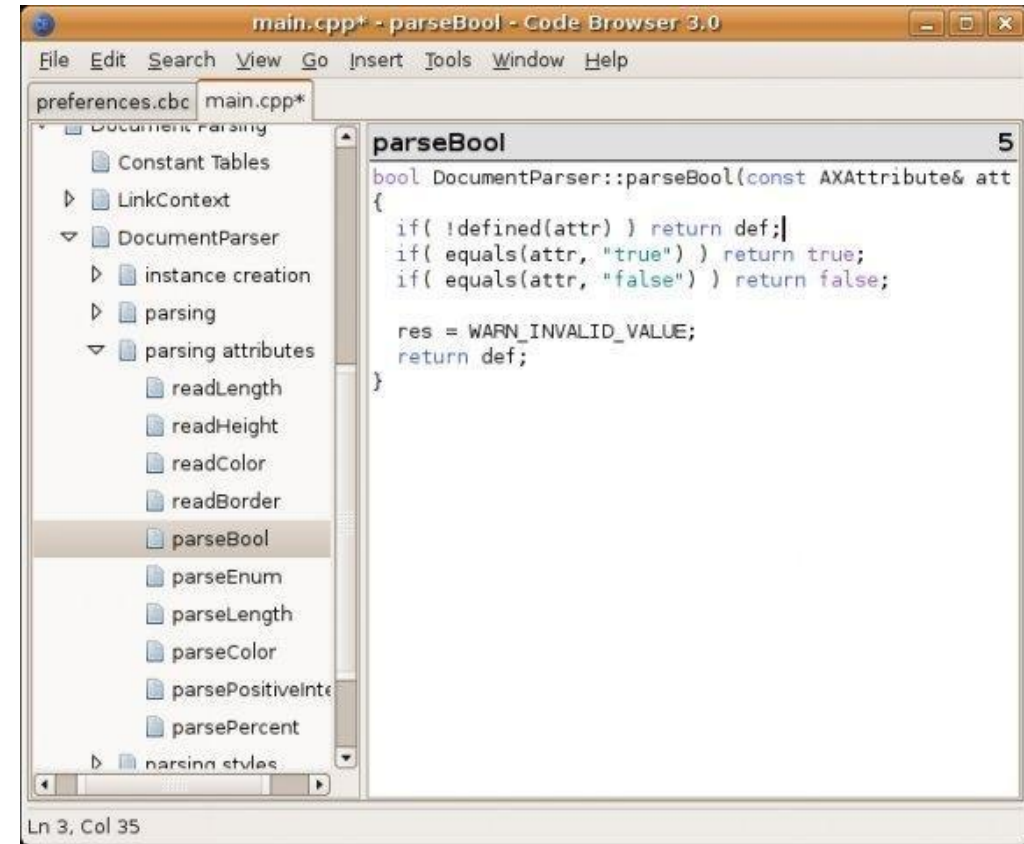
Techniques for Maintenance

Program comprehension

- Technical issue: limited understanding
- Unique activities: program understanding
- Techniques: program comprehension

“Programmers spend considerable time reading and understanding programs in order to implement changes.”

- Code Browser (folding text editor)
- Clear and concise documentation



Techniques for Maintenance

Re-engineering

“Reengineering is defined as the examination and alteration of software to reconstitute it in a new form”

- Used to replace aging or legacy software
- Used to improve program structure and maintainability
- Use refactoring to reorganize a program without changing its behavior

Techniques for Maintenance

Reverse Engineering

“Reverse engineering is the process of analyzing software to identify the software’s components and their inter-relationships and to create representations of the software in another form or at higher levels of abstraction.”

- Produce call graphs and control flow graphs from source code
- Redocumentation
- Design recovery
- Data reverse engineering

Techniques for Maintenance

Migration

“During software’s life, it may have to be modified to run in different environments.”

- Determine the actions needed to accomplish the migration
- Develop and document the steps required
- Migration plan
 - Migration requirements
 - Migration tools
 - Conversion of product and data
 - Execution
 - Verification
 - Support

Techniques for Maintenance

Retirement

“Once software has reached the end of its useful life, it must be retired.”

- Analysis is performed to make the decision to retire
- Retirement plan
 - Analysis
 - Retirement requirements
 - Impact
 - Replacement
 - Schedule
 - Effort
 - Accessibility of archived data

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

PRESS RECORD

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Maintenance - Tools and Example

COMPX₃₄₁

Overview

- Maintenance Tools
- Maintenance Example

Maintenance Tools

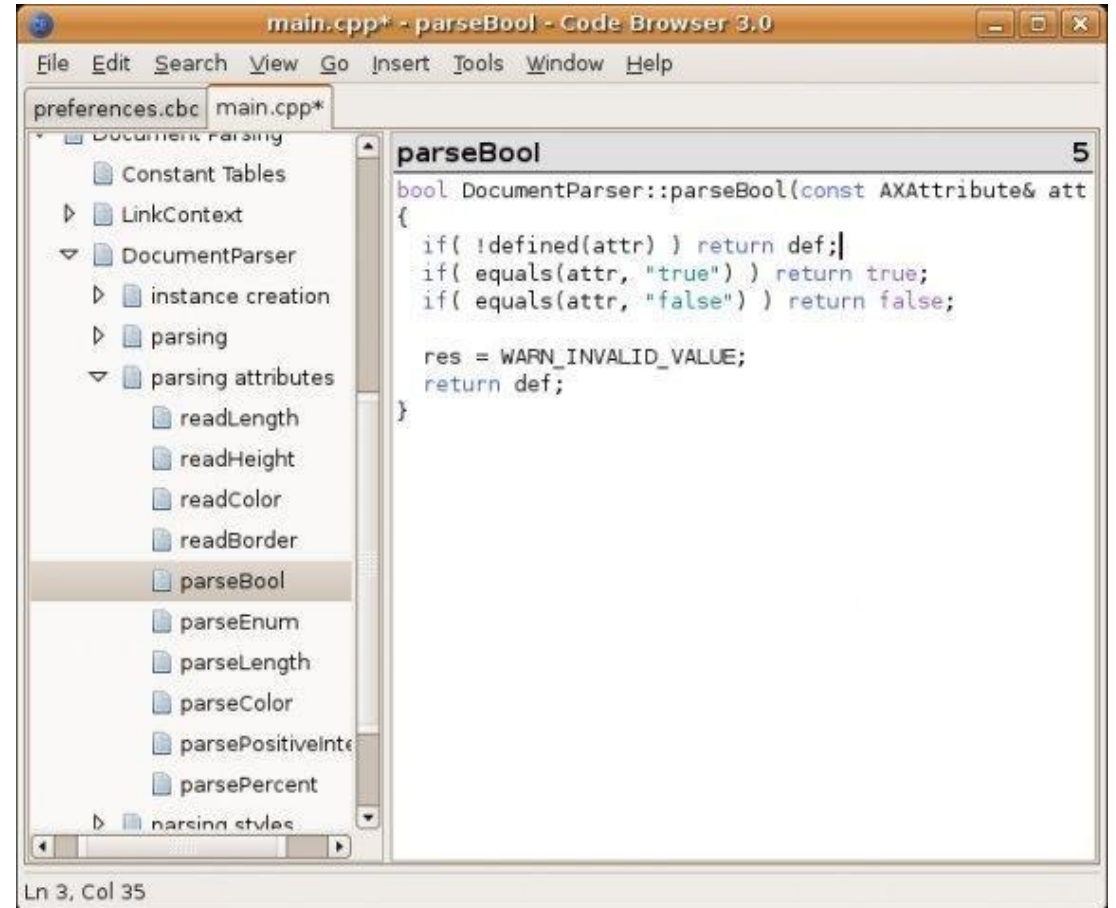
Maintenance Tools

- Code browsers
- Program slicers
- Static analyzers
- Dynamic analyzers
- Data flow analyzers
- ...

Maintenance Tools

Code browser

- A text editor or IDE that includes a folding layout
- Structures source code



Maintenance Tools

Program slicers

- Slicing is the computation of a set of statements
- By creating a slice, you eliminate all other parts of the program that are not currently of interest
- You can then focus on the code that is of interest to you (usually because there is an error occurring)
- Programmer clicks on a statement, then the program slicer tool highlights all statements that might affect the statement of interest (i.e. creates a slice)

Maintenance Tools

Static analyzers

- Static Code Analysis is the analysis of a system without running it.
 - Data flow analysis
 - Control flow graph
 - Taint analysis
 - Lexical analysis
 - ...
- https://owasp.org/www-community/controls/Static_Code_Analysis

Maintenance Tools

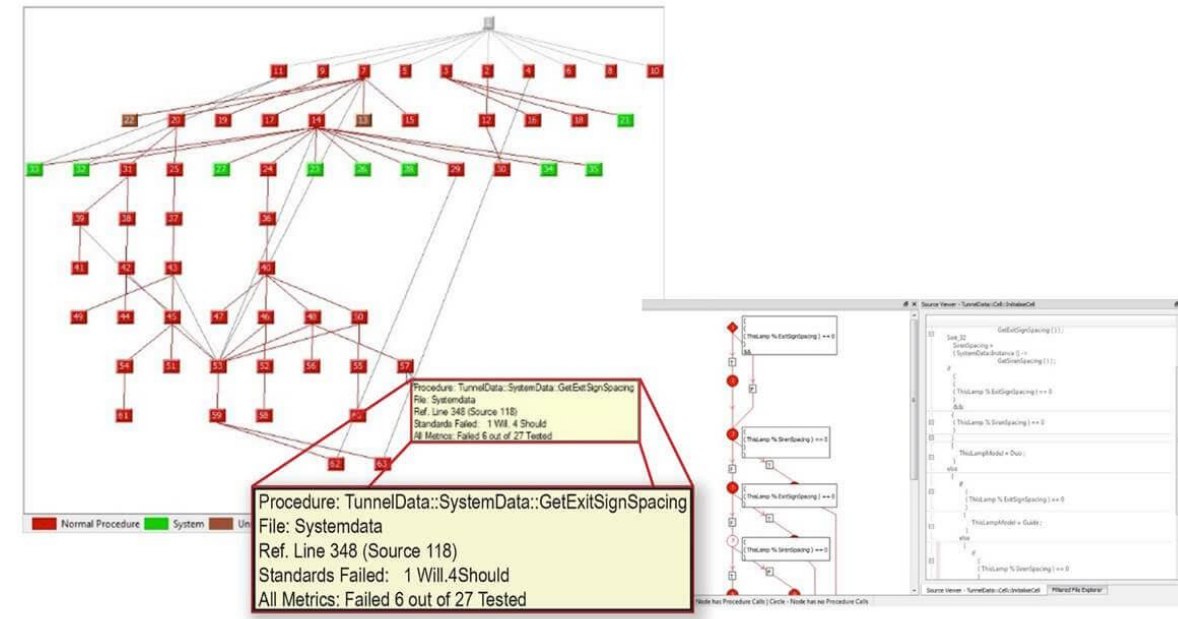
Dynamic analyzers

- Dynamic Code Analysis is the analysis of a system while it is running.
 - Compile time and run time vulnerabilities
 - Configuration errors
- Dynamic Analysis Tools can test
 - SQL queries
 - Long input strings
 - Negative and large positive numbers
 - Unexpected input data
- <https://www.checkpoint.com/cyber-hub/cloud-security/what-is-dynamic-code-analysis/>

Maintenance Tools

Data flow analyzers

- Used to gather information about values that are calculated at different points in a system
 - Control flow graph
 - Setup dataflow equations for each node in the graph
 - Solve them by repeatedly calculating output from given input



- <https://clang.llvm.org/docs/DataFlowAnalysisIntro.html>
- <https://lra.com/capabilities/data-flowcontrol-flow-analysis/>

Maintenance Example

Maintenance Example

"We would like to create a spell checker application. The application will take a word and either confirm that it is spelt correctly, or confirm that it is not spelt correctly. Input words may include punctuation, but cannot include numerical characters"

Maintenance Example

"A user has identified a bug in the code. The input "time()" is expected to result in the output true. Instead, it results in the output false."

"Your company has decided that the system code needs to be re-engineered for greater maintainability"

"The client would like you to add the ability to handle strings with numeric values"

Maintenance Example

- For each:
 1. Problem/modification request
 2. Problem/modification analysis
 - Determine the maintenance category (Preventative, perfective, corrective, adaptive)
 - Conduct an *informal* Impact Analysis evaluation
 3. Modification request acceptance/rejection
 - Determine whether to accept the request
 4. Modification implementation
 - Implement the modification
 5. Maintenance review / acceptance
 - Testing and/or code review

Maintenance Example

"A user has identified a bug in the code. The input "time()" is expected to result in the output true. Instead, it results in the output false."

Maintenance Example

1. Problem/modification request

A user has identified a bug in the code. The input "time()" is expected to result in the output true. Instead, it results in the output false.

Problem request: investigate and fix the bug that results in the input "time()" returning false.

2. Problem/modification analysis

Maintenance category: ?

... remember

Software Maintenance Categories

	Correction Same requirements	Enhancement Changes requirements and/or changed environment
Proactive To prevent failure	Preventive Bugs are there but not surfacing ... yet!	Perfective New features or documentation; better performance; etc.
Reactive Because of one or more reported failures	Corrective a.k.a. Repairs Special case: emergency	Adaptive Changed/changing environment (e.g., updated OS, middleware, etc.)

Maintenance Example

1. Problem/modification request

A user has identified a bug in the code. The input "time()" is expected to result in the output true. Instead, it results in the output false.

Problem request: investigate and fix the bug that results in the input "time()" returning false.

2. Problem/modification analysis

Maintenance category: Corrective

Maintenance Example

1. Problem/modification request

A user has identified a bug in the code. The input "time()" is expected to result in the output true. Instead, it results in the output false.

Problem request: investigate and fix the bug that results in the input "time()" returning false.

2. Problem/modification analysis

Maintenance category: Corrective

Impact Analysis:

- Verify the problem:
- Implementation option:
- Effort:
- Resources:

Maintenance Example

1. Problem/modification request

A user has identified a bug in the code. The input "time()" is expected to result in the output true. Instead, it results in the output false.

Problem request: investigate and fix the bug that results in the input "time()" returning false.

2. Problem/modification analysis

Maintenance category: Corrective

Impact Analysis:

- Verify the problem:
- Implementation option:
- Effort:
- Resources:

Let's take a look ...

Maintenance Example

1. Problem/modification request

A user has identified a bug in the code. The input "time()" is expected to result in the output true. Instead, it results in the output false.

Problem request: investigate and fix the bug that results in the input "time()" returning false.

2. Problem/modification analysis

Maintenance category: Corrective

Impact Analysis:

- Verify the problem: only a subset of punctuation is currently covered
- Implementation option: use *punct* regex to cover a larger group of punctuation characters
- Effort: alter one line of code, no new knowledge required
- Resources: less than 15 minutes of a maintainer's time (what about re-running tests? etc.)

Maintenance Example

3. Modification request acceptance/rejection

Decision: Accepted

4. Modification implementation

- Implement the modification

5. Maintenance review / acceptance

- Re-run regression tests
- Re-run acceptance tests
- Submit to code review
- On approval, submit to Git

Maintenance Example

"Your company has decided that the system code needs to be re-engineered for greater maintainability"

Maintenance Example

1. Problem/modification request

Your company has decided that the system code needs to be re-engineered for greater maintainability

Modification request: Use code and solution quality techniques to modularize the code for better maintainability.

2. Problem/modification analysis

Maintenance category: ?

... remember

Software Maintenance Categories

	Correction Same requirements	Enhancement Changes requirements and/or changed environment
Proactive To prevent failure	Preventive Bugs are there but not surfacing ... yet!	Perfective New features or documentation; better performance; etc.
Reactive Because of one or more reported failures	Corrective a.k.a. Repairs Special case: emergency	Adaptive Changed/changing environment (e.g., updated OS, middleware, etc.)

Maintenance Example

1. Problem/modification request

Your company has decided that the system code needs to be re-engineered for greater maintainability

Modification request: Use code and solution quality techniques to modularize the code for better maintainability.

2. Problem/modification analysis

Maintenance category: Preventative

Maintenance Example

1. Problem/modification request

Your company has decided that the system code needs to be re-engineered for greater maintainability

Modification request: Use code and solution quality techniques to modularize the code for better maintainability.

2. Problem/modification analysis

Maintenance category: Preventative

Impact Analysis:

- Verify the problem:
- Implementation option:
- Effort:
- Resources:

Maintenance Example

1. Problem/modification request

Your company has decided that the system code needs to be re-engineered for greater maintainability

Modification request: Use code and solution quality techniques to modularize the code for better maintainability.

2. Problem/modification analysis

Maintenance category: Preventative

Impact Analysis:

- Verify the problem:
- Implementation option:
- Effort:
- Resources:

Let's take a look ...

Maintenance Example

1. Problem/modification request

Your company has decided that the system code needs to be re-engineered for greater maintainability

Modification request: Use code and solution quality techniques to modularize the code for better maintainability.

2. Problem/modification analysis

Maintenance category: Preventative

Impact Analysis:

- Verify the problem: Code is contained in one class. Should design a new modularized solution.
- Implementation option: re-engineer code to adhere to newly created component and class diagram
- Effort: alter all lines of code, new knowledge?
- Resources: significant amount of the maintainer's time, a full re-engineer of the entire system, plus re-running of all tests. May need a staged approach so as to leave the system live during update.

Maintenance Example

3. Modification request acceptance/rejection

Decision: ???

4. Modification implementation

- Implement the modification

5. Maintenance review / acceptance

- Re-run regression tests
- Re-run acceptance tests
- Submit to code review
- On approval, submit to Git

Maintenance Example

"The client would like you to add the ability to handle strings with numeric values"

Maintenance Example

1. Problem/modification request

The client would like you to add the ability to handle strings with numeric values

Modification request: Added feature. The system must have the ability to strip numeric values from strings.

2. Problem/modification analysis

Maintenance category: ?

... remember

Software Maintenance Categories

	Correction Same requirements	Enhancement Changes requirements and/or changed environment
Proactive To prevent failure	Preventive Bugs are there but not surfacing ... yet!	Perfective New features or documentation; better performance; etc.
Reactive Because of one or more reported failures	Corrective a.k.a. Repairs Special case: emergency	Adaptive Changed/changing environment (e.g., updated OS, middleware, etc.)

Maintenance Example

1. Problem/modification request

The client would like you to add the ability to handle strings with numeric values

Modification request: Added feature. The system must have the ability to strip numeric values from strings.

2. Problem/modification analysis

Maintenance category: Perfective

Maintenance Example

1. Problem/modification request

The client would like you to add the ability to handle strings with numeric values

Modification request: Added feature. The system must have the ability to strip numeric values from strings.

2. Problem/modification analysis

Maintenance category: Perfective

Impact Analysis:

- Verify the problem:
- Implementation option:
- Effort:
- Resources:

Maintenance Example

1. Problem/modification request

The client would like you to add the ability to handle strings with numeric values

Modification request: Added feature. The system must have the ability to strip numeric values from strings.

2. Problem/modification analysis

Maintenance category: Perfective

Impact Analysis:

- Verify the problem:
- Implementation option:
- Effort:
- Resources:

Let's take a look ...

Maintenance Example

1. Problem/modification request

The client would like you to add the ability to handle strings with numeric values

Modification request: Added feature. The system must have the ability to strip numeric values from strings.

2. Problem/modification analysis

Maintenance category: Perfective

Impact Analysis:

- Verify the problem: the system currently disregards strings with numeric values. Instead, the new feature will allow the system to strip numeric values from strings.
- Implementation option: create a new method called stripNumeric that will strip numeric values from a string. Update the source code to use this method rather than disregarding strings with numeric values
- Effort: alter 5-10 lines of code, any new knowledge?
- Resources: less than an hour of a maintainer's time (what about re-running tests? etc.)

Maintenance Example

3. Modification request acceptance/rejection

Decision: Accepted

4. Modification implementation

- Implement the modification

5. Maintenance review / acceptance

- Re-run regression tests
- Re-run acceptance tests
- Submit to code review
- On approval, submit to Git

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

PRESS RECORD

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Before we start ...

- Guest speaker this Friday
- Please come along in person



Liam Clow
DevOps Engineer
Hamilton City Council

DevOps – an Introduction

COMPX₃₄₁

Overview

- What is DevOps?
- What is SRE?
- DevOps/SRE and the Cloud

What is DevOps?

What is DevOps?

- Dev
 - Development teams focus on the software product
- Ops
 - System administrator teams (sysadmin) focus on operations
 - i.e. Assembling/deploying software to run a service
 - i.e. Responding to events/updates/traffic/etc.
- DevOps:
 - Run Ops as a software engineering project
 - Design/build software to automate sysadmin/ops tasks
 - Mixed skills needed!

What is SRE?

What is Site Reliability Engineering?

“By design, it is crucial that SRE teams are focused on engineering. Without constant engineering, operations load increases and teams will need more people just to keep pace with the workload. Eventually, a traditional ops-focused group scales linearly with service size: if the products supported by the service succeed, the operational load will grow with traffic. That means hiring more people to do the same tasks over and over again.” – *Benjamin Treynor Sloss, Google SRE Book*

What is Site Reliability Engineering?

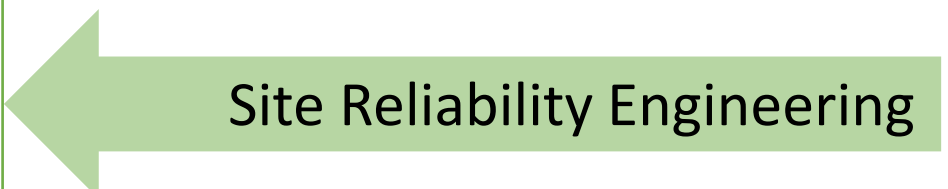
“To avoid this fate, the team tasked with managing a service needs to code or it will drown.”

“... the SRE team should end up with very little operational load and almost entirely engage in development tasks, because the service basically runs and repairs itself: we want systems that are **automatic**, not just **automated**. ”

– Benjamin Treynor Sloss, *Google SRE Book*

What is Site Reliability Engineering?

- Dev
 - Development teams focus on the software product
- Ops
 - System administrator teams (sysadmin) focus on operations
 - i.e. Assembling/deploying software to run a service
 - i.e. Responding to events/updates/traffic/etc.
- DevOps:
 - Run Ops as a software engineering project
 - Design/build software to automate sysadmin/ops tasks
 - Mixed skills needed!



Site Reliability Engineering

What is Site Reliability Engineering?

SRE is DevOps, with some caveats

- SRE is DevOps done by Software Engineers
- SRE is DevOps with a set of core tenets

Tenets of SRE

Site Reliability Engineering has eight tenets:

1. Availability
2. Latency
3. Monitoring
4. Emergency response
5. Change management
6. Capacity planning
7. Efficiency
8. Performance

And SRE teams are responsible for upholding these tenets

Tenet 1: availability

“Availability refers to the percentage of time that the infrastructure, system, or solution remains operational under normal circumstances in order to serve its intended purpose.” — <https://www.bmc.com/blogs/reliability-vs-availability/>

- SRE: 100% availability is the wrong target
- Instead:
 - What level of availability will the users be happy with?
 - What alternatives are available ?
 - What happens to usage of the product at different availability levels?
- Can use remaining percentage as ‘error budget’
 - i.e. Error budget = $1 - \text{availability}$. E.g. 0.999 availability leaves 0.001 (or 0.1%) error budget
 - i.e. can spend the error budget on innovations and feature velocity

Tenet 2: latency

- Humans add latency
- Imagine two systems
 - System A: relies heavily on human decisions and interventions
 - System B: able to avoid some human interventions
 - Even if System B experiences more failures than System A, it will have higher availability because it does not rely on human decisions and interventions as heavily, i.e. Avoids high levels of latency
- On-call playbooks
 - No system can avoid all human interventions
 - In these cases, SRE relies on on-call playbooks to help reduce latency
 - Playbooks are created by thinking and recording best practices ahead of time

Tenet 3: monitoring

- A way to keep track of a system's health and availability
- Common approach:
 - Watch for a trigger
 - Send an email alert to human
 - Wait for decision from human
- SRE approach:
 - Alerts
 - Human needs to take action immediately
 - Ticketing
 - Human need to take action, but not immediately
 - Logging
 - System handles the information

Tenet 4: emergency response

- MTTF: mean time to failure
- MTTR: mean time to repair
- We can evaluate the effectiveness of emergency responses by considering how long it will take to bring the system back to health
- Note: human latency will play a role in this

Tenet 5: change management

“SRE has found that roughly 70% of outages are due to changes in a live system. ”

– Benjamin Treynor Sloss, *Google SRE Book*

- SRE uses automation to:
 - Implement progressive rollouts
 - Detect problems (quickly and accurately)
 - Roll back changes when problems have been detected
- This minimizes the number of operations that are exposed to ‘bad’ changes
- Automation also removes humans from the loop (ish), which mitigates problems of:
 - Fatigue
 - Familiarity/contempt
 - Inattention to highly repetitive tasks

Tenet 6: Capacity planning

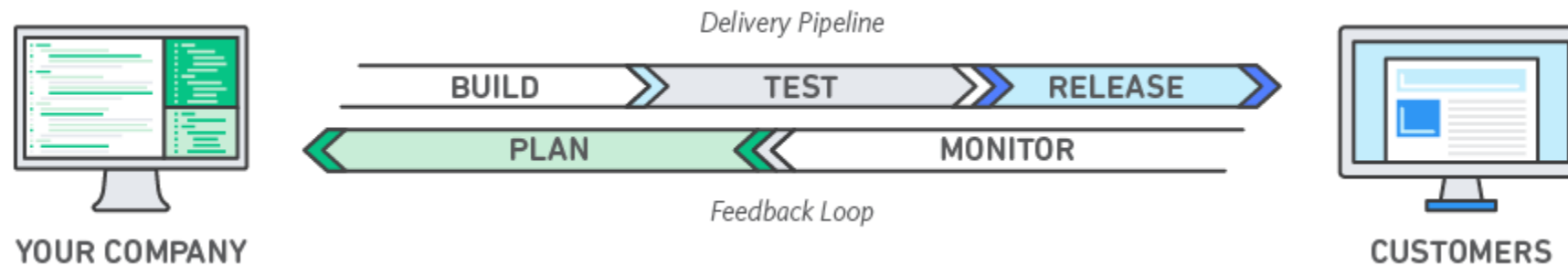
- Ensuring there is sufficient capacity and redundancy for future demand
- Ensuring that the capacity is in place by the time it is needed (before, not at or after)
- Should consider:
 - Organic growth: natural product adoption
 - Inorganic growth: growth stemming from feature launches, marketing, etc.
- Three capacity planning steps:
 1. Demand forecast for organic growth, extending beyond the lead time
 2. Incorporation of inorganic events
 3. Regular load testing
- Note, since SRE is in charge of capacity planning, they should also be in charge of provisioning

Tenet 7 & 8: Efficiency and Performance

- Resource use involves
 - Demand (load)
 - Capacity
 - Efficiency
- SREs predict demand and capacity, and can modify the software/services for efficiency.
- SREs and developers should monitor and modify services to improve performance
 - i.e. Predict capacity
 - i.e. Add capacity (provisions)
 - i.e. Improve efficiency

Introducing DevOps/SRE on the cloud

What does DevOps look like on the cloud?



What does DevOps look like on the cloud?

- Speed
- Rapid delivery
- Reliability
- Scale

What does DevOps look like on the cloud?

- Continuous integration
- Continuous delivery
- Micro-services
- Infrastructure as code
- Monitoring and logging

What does DevOps look like on the cloud?

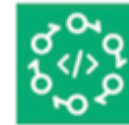
Cloud-providers provide us with DevOp services



Software Release Workflows
AWS CodePipeline



Build and Test Code
AWS CodeBuild



Unified CI/CD Projects
AWS CodeStar



Deployment Automation
AWS CodeDeploy

What does DevOps look like on the cloud?

Cloud-providers provide us with DevOp services



Azure Pipelines



Azure Artifacts



Azure Test Plans



Azure Boards



Azure Repos

What does DevOps look like on the cloud?

Cloud-providers provide us with DevOp services



Google Cloud Deploy



Cloud Build



Operations Suite



Artifact Registry



Binary Authorization

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

PRESS RECORD

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Before we start ...

- Guest speaker this Friday
- Please come along in person



Liam Clow
DevOps Engineer
Hamilton City Council

DevOps – CI/CD Pipelines

COMPX₃₄₁

Overview

- What is CI/CD?
- What are CI/CD pipelines?
- CI/CD pipeline example

What is CI/CD

What is CI/CD?

“CI/CD is a method to frequently deliver apps to customers by introducing automation into the stages of app development. ”

<https://www.redhat.com/en/topics/devops/what-is-ci-cd>

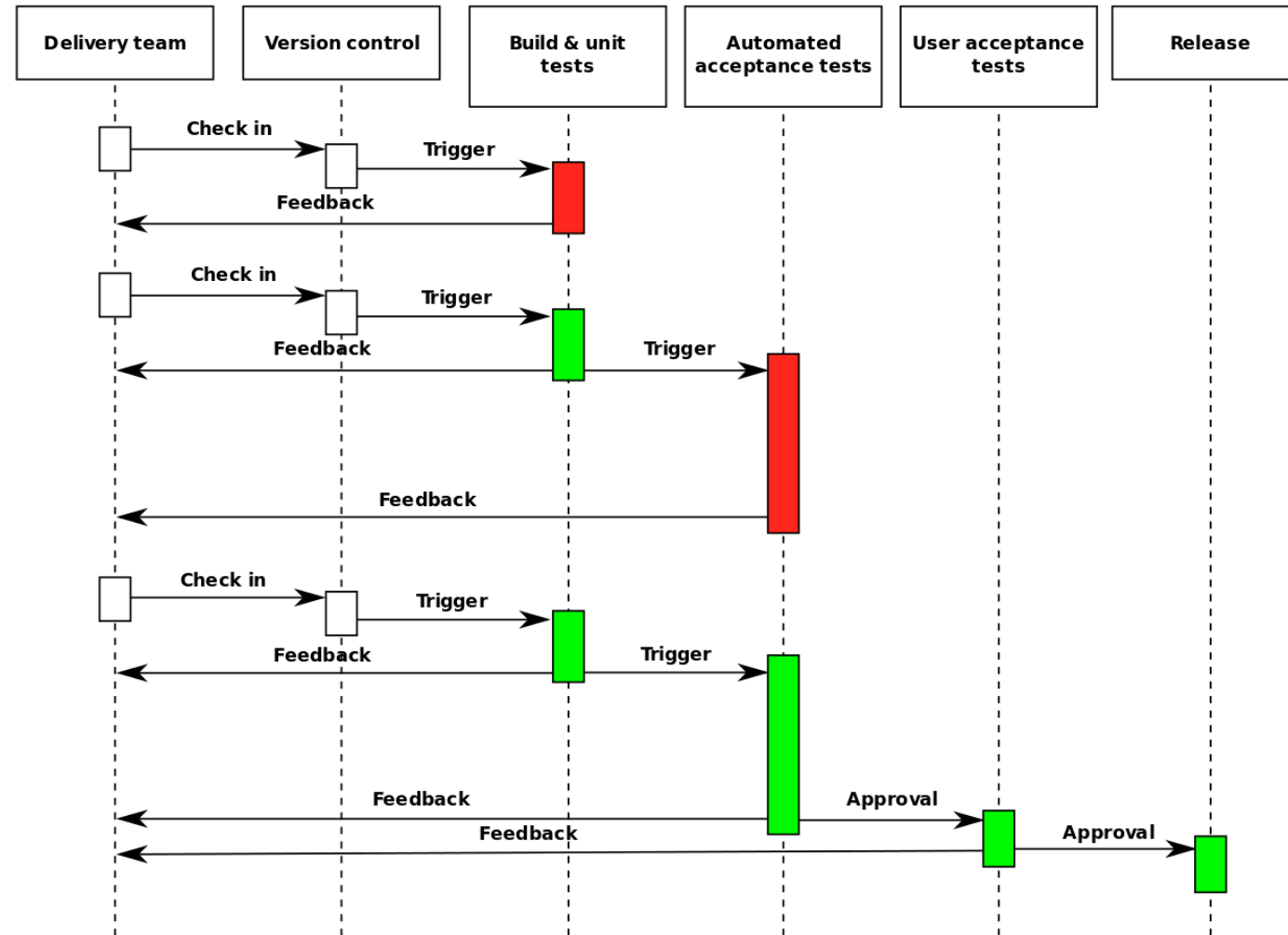
CI/CD stands for Continuous Integration and Continuous Delivery/Continuous Deployment.

What is CI/CD?

Continuous Integration / Continuous Delivery / Continuous Deployment

- Continuous integration
 - Frequent merging of separate dev/team code several times a day
- Continuous delivery/Continuous Deployment
 - Aim for constant system availability
 - Aims to increase the rate of producing new versions
 - Aims to shorten the dev/testing time
 - Automates processes

CI/CD example



CI/CD Pipelines

CI/CD Pipelines

- A CI/CD pipeline is a series of steps that are taken/performed in order to deliver software
- CI/CD pipelines focus on improving delivery using a DevOps or SRE approach
- Note, it is possible to manually execute each step in a CI/CD pipeline. However, the true value of CI/CD pipelines is their automation!

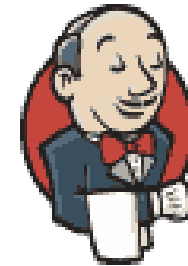
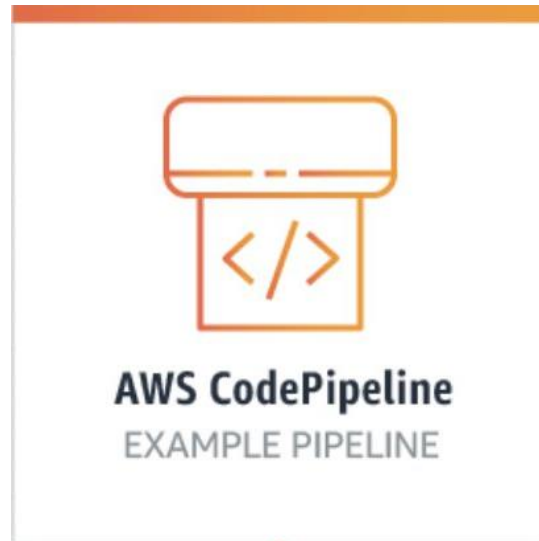
CI/CD Pipelines

- Typical CI/CD pipeline stages include:
 - Build (compiling the application)
 - Test (code testing, i.e. unit tests)
 - Release (committing to a repo)
 - Deploy (deployed to production)

CI/CD Pipelines



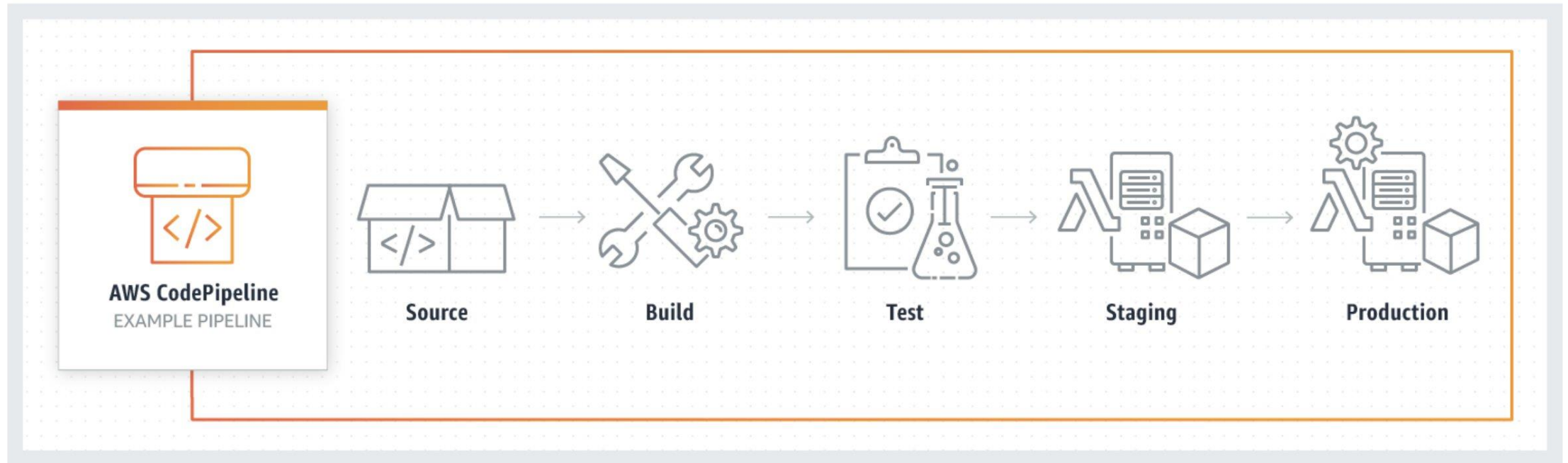
CI/CD Pipelines



Jenkins

CI/CD Pipeline Example

CI/CD pipelines in AWS



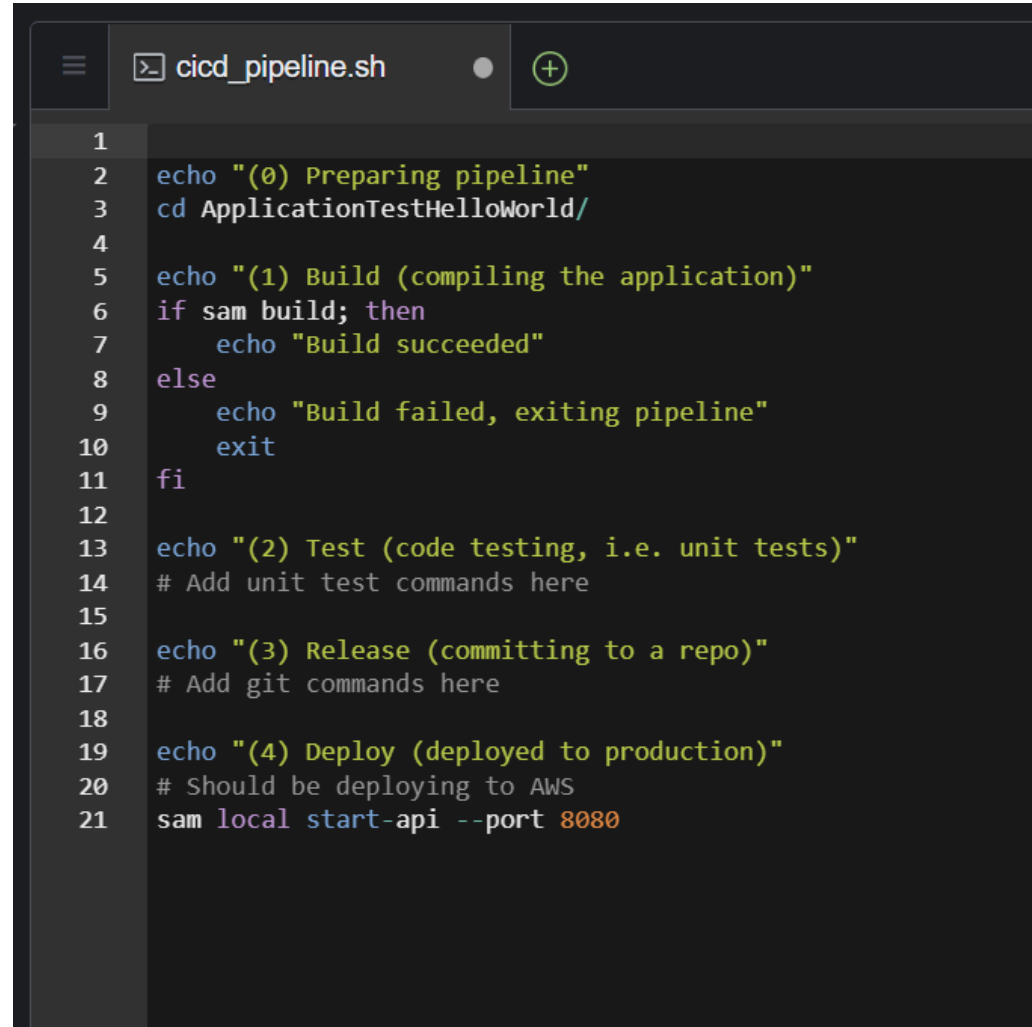
CI/CD pipelines for the command line (Linux)

You can also write a bash script

What is a bash script?

- A plain text file that contains a set of CLI commands
- The file extension is .sh
- For example ...

CI/CD pipelines for the command line (Linux)



```
1
2 echo "(0) Preparing pipeline"
3 cd ApplicationTestHelloWorld/
4
5 echo "(1) Build (compiling the application)"
6 if sam build; then
7     echo "Build succeeded"
8 else
9     echo "Build failed, exiting pipeline"
10    exit
11 fi
12
13 echo "(2) Test (code testing, i.e. unit tests)"
14 # Add unit test commands here
15
16 echo "(3) Release (committing to a repo)"
17 # Add git commands here
18
19 echo "(4) Deploy (deployed to production)"
20 # Should be deploying to AWS
21 sam local start-api --port 8080
```

CI/CD pipelines for the command line (Windows)

You can also write a batch script (Windows)

What is a batch script?

- A plain text file that contains a set of CLI commands
- The file extension is .bat
- For example ...

CI/CD pipelines for the command line (Windows)

```
run_empty.bat x
1  @echo off
2
3  echo:
4  echo (0) Preparing pipeline
5
6  echo:
7  echo (1) Build (compiling the application)
8  javac -cp "junit-platform-console-standalone-1.8.2.jar" *.java
9  IF %ERRORLEVEL% NEQ 0 (
10     echo Build failed, exiting pipeline
11     set /p DUMMY=Hit ENTER to finish...
12     EXIT /B
13 ) ELSE (
14     echo Build successful
15 )
16
17 echo:
18 echo (2) Test (running the test suite)
19 rem Add unit test commands here
20
21 echo:
22 echo (3) Release (committing to repo)
23 rem Add git commands here
24
25 echo:
26 echo (4) Deploy (deployed to production)
27 rem Add run commands here
28
29 echo:
30 set /p DUMMY=Hit ENTER to finish...
```

CI/CD pipelines for the command line (Linux/Windows)

You can also write a bash/batch script

What pipeline should (at minimum) include

1. Some form of preparation
2. Build (compiling the software)
3. Test (e.g. running regression tests)
4. Release (committing to a repository)
5. Deploy (optional, depends on the pipeline)

CI/CD pipelines for the command line (Windows)

- Okay, let's give it a try ...

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

COMPX341-23A

Advanced Software Engineering Design

Week 7
Lecture 17

Unit testing example

“Create a class `IntSet` which provides set operations for sets of integers. You should be able to add to a set, test whether an element is in a set, delete from a set, form the intersection and unions of sets.”

Unit testing example

Look at implementation and a “test” run with printing to “debug”...

Unit testing example

- Alternatively, write a test suite...
- Units:
 - Write unit tests to test the method that adds elements to a set
 - Write unit tests to test the method that deletes elements from a set
 - Write unit tests to test the method that forms intersections of sets
 - Write unit tests to test the method that forms unions of sets

Unit testing example

```
// Arrange
IntSet a = new IntSet(5)

// Act
a.add(1); a.add(2); a.add(3);

// Assert
a.hasElement(4)
```

Unit testing example

```
// Arrange  
IntSet a = new IntSet(5)  
  
// Act  
a.add(1); a.add(2); a.add(3);  
  
// Assert  
a.hasElement(3)
```

Unit testing example

```
// Arrange  
IntSet a = new IntSet(5)  
IntSet b = new IntSet(5)  
  
// Act  
a.add(1) a.add(2) a.add(3) b.add(3)  
IntSet c = a.intersection(b)  
  
// Assert  
c.hasElement(3)
```

JUnit

JUnit

- A very common package for writing, organising, and running tests in Java
- You will need to set up your environment for JUnit (sometimes)
- I'm using JUnit4 since it's simpler than the latest version but does everything we need
- Just use an editor and a terminal (IDEs can be unwieldy and obscure the simple ideas)
- Good introduction, which I'll work through (and add to):

<https://github.com/junit-team/junit4/wiki/Getting-started>

Writing tests with JUnit

A JUnit test method has three requirements:

1. It isn't static or private
2. It can't return anything (must be void)
3. A test method must be annotated with `@Test`

Writing tests with JUnit

In JUnit, annotations are used to specify when methods should run during testing.

Some other common annotations include:

- **@Before**: execute this method before each test
- **@After**: execute this method after each test

Writing tests with JUnit

In JUnit, values are evaluated using “assertions”.

Some common assertions include:

- **assertEquals(x, y)**: asserts that x and y are equal
- **assertNotEquals(x, y)**: asserts that x and y are not equal
- **assertTrue(boolean condition)**: asserts that the supplied condition is true
- **assertFalse(boolean condition)**: asserts that the supplied condition is not true
- **assertNotNull(Object actual)**: asserts that actual is not null
- ... and lots more!

<https://github.com/junit-team/junit4/wiki/Assertions>

For example ...

Assume that we have the following class:

```
public class HelloWorld {  
  
    public String helloWorld(){  
        return "Hello World!";  
    }  
}
```

We want to test the “helloWorld” method to determine if it returns the correct string.

Example Test

```
import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class HelloWorldTest {
    @Test
    public void helloWorldTest() {
        HelloWorld t = new HelloWorld();
        assertEquals(t.helloWorld(), "Hello World!");
    }
}
```

Examples.....see the files attached to this week on Moodle

TestJUnit

Calculator

IntSet, TestTrivIntSet and TestIntSet